A FINAL-YEAR PROJECT REPORT ON

ScamShield

Pre-Transaction Scam Detection and Proportional Intervention
for UPI-Style Payments using a Hybrid Rules + Machine-Learning Engine

Submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering / Electronics and Communication Engineering

Submitted by

Ansh	23BCS016
Harshit	23BCS054
Vijender	23BCS132
Sourabh	23BEC055

Under the guidance of

Dr. Anushree Kini

Indian Institute of Information Technology Dharwad

Dharwad, Karnataka – 580009

Academic Year 2026–27

CERTIFICATE

This is to certify that the project report entitled **"ScamShield: Pre-Transaction Scam Detection and Proportional Intervention for UPI-Style Payments using a Hybrid Rules + Machine-Learning Engine"** is a bona fide record of the work carried out by

Name	Roll Number
Ansh	23BCS016
Harshit	23BCS054
Vijender	23BCS132
Sourabh	23BEC055

in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology at the Indian Institute of Information Technology Dharwad during the academic year 2026–27, under my supervision and guidance. To the best of my knowledge, the matter embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Dr. Anushree Kini
Project Guide
IIIT Dharwad

Date: _____

Head of the Department
Department of Computer Science and Engineering
IIIT Dharwad

Date: _____

Internal Examiner
Name and signature

External Examiner
Name and signature

DECLARATION

We hereby declare that the project report entitled **"ScamShield: Pre-Transaction Scam Detection and Proportional Intervention for UPI-Style Payments using a Hybrid Rules + Machine-Learning Engine"**, submitted to the Indian Institute of Information Technology Dharwad in partial fulfilment of the requirements for the degree of Bachelor of Technology, is a record of original work carried out by us under the guidance of Dr. Anushree Kini.

The work reported here has not been submitted, in part or in full, to any other university or institute for the award of any degree or diploma. Wherever we have used material from other sources — publications, government data, software libraries or online documentation — it has been duly acknowledged and cited in the References. All users, payments, balances and fraud cases described in this report are **synthetic**; no real customer data, bank system or payment network was used.

Ansh (23BCS016)	Harshit (23BCS054)
Vijender (23BCS132)	Sourabh (23BEC055)

Place: Dharwad
Date: _____

ACKNOWLEDGEMENT

We would like to thank our project guide, **Dr. Anushree Kini**, for her guidance and feedback throughout this project.

We also thank the Director, the Head of the Department and the faculty of IIIT Dharwad for giving us the opportunity and the facilities to work on this project.

Thanks to our friends and families for their support, and to the people who took the time to fill in our questionnaire.

Finally, this project would not have been possible without the open-source tools it is built on, including Next.js, Express, FastAPI, scikit-learn, PostgreSQL, Redis, BullMQ, Prisma and Playwright.

Ansh, Harshit, Vijender, Sourabh

ABSTRACT

UPI handled more than 24,000 crore transactions in FY2025-26, and fraud on it has grown with it. Most of this fraud is not hacking. The victim is talked into making the payment themselves by a fake bank or KYC agent, a fake police officer, a job offer or a remote-access app. PINs and OTPs do not help here, and once the money is sent it is hard to get back.

In this project we built **ScamShield**, a system that checks a UPI-style payment for scam risk before the money moves. For each payment it computes 20 behavioural features, such as how the amount compares with the user's last 30 days, how old the receiving account is, and how many different people paid that account today. Eleven rules and a calibrated gradient boosting model each give a score, and the two are blended into a 0 to 100 risk score with a LOW, MEDIUM or HIGH band and reasons in plain language. LOW payments go through with no interruption. MEDIUM payments show a warning and three short questions about the situation, such as whether someone is on the phone asking them to pay. HIGH payments get a 10-second pause and the payer has to type the UPI ID again. Analysts see alerts live, and when they confirm a scam account it is blocked for all users.

The system is built with Next.js, Express, FastAPI, PostgreSQL and Redis, and runs with Docker. Since real fraud data was not available, we wrote a synthetic data generator with realistic genuine and scam behaviour and tested on a later time period of 45,387 payments. At the same false alarm rate as a rules-only system, the hybrid caught 95.6 % of scams against 73.3 %. ML alone ranked payments slightly better than the hybrid (PR-AUC 0.946 against 0.933), and the report discusses this trade-off. The 95th percentile response time was about 110 ms, and all 341 automated tests pass. All results are on synthetic data, and testing on real data is the main next step.

Keywords: UPI, scam detection, authorised push payment fraud, gradient boosting, calibration, security warnings.

TABLE OF CONTENTS

Certificate ..... **ii**

Declaration ..... **iii**

Acknowledgement ..... **iv**

Abstract ..... **v**

List of Figures ..... **ix**

List of Tables ..... **x**

List of Abbreviations ..... **xi**

Chapter 1 Introduction **1**

1.1 Background 1

1.2 Motivation 1

1.3 Objectives 2

1.4 Scope 2

1.5 Organisation of the Report 2

Chapter 2 Problem Definition **3**

2.1 Problem Statement 3

2.2 Why It Is a Real Problem 3

2.3 Root Causes 4

2.4 Who Is Affected 5

Chapter 3 User Research **6**

3.1 User Personas 6

3.2 Questionnaire 7

3.3 Responses and Key Insights 7

3.4 User Journey Map 8

Chapter 4 Market and Competitive Analysis **9**

4.1 Market Size 9

4.2 Existing Solutions 9

4.3 Feature Comparison 10

4.4 SWOT Analysis 10

4.5 Unique Value Proposition 11

Chapter 5 Proposed Solution **12**

5.1 Overview 12

5.2 Key Features 12

5.3 Business Model 13

5.4 Go-to-Market Strategy 13

Chapter 6 Requirements Specification **14**

6.1 Functional Requirements 14

6.2 Non-Functional Requirements 15

6.3 Hardware and Software Requirements 15
6.4 Constraints and Assumptions 15

Chapter 7 Policies, Legal and Compliance 17
7.1 Privacy and Data Protection 17
7.2 Terms of Service and Consent 18
7.3 Security Policies 18
7.4 Accessibility and Ethics 18
7.5 Payments Sector Rules 19

Chapter 8 System Design and Technical Details 20
8.1 System Architecture 20
8.2 Technology Stack 22
8.3 Data Flow Diagrams 22
8.4 UML Diagrams 23
8.5 Database Schema 27
8.6 APIs and Integrations 28
8.7 Algorithms and Business Rules 28
8.8 Synthetic Data Generation 30

Chapter 9 Implementation 31
9.1 What We Built 31
9.2 Development Method 32
9.3 Team Roles 32
9.4 Screenshots 33

Chapter 10 Testing and Results 37
10.1 Testing Strategy 37
10.2 Test Cases 37
10.3 Model Results 38
10.4 Performance 40
10.5 User Feedback 40

Chapter 11 Challenges and Limitations 42
11.1 Problems We Faced 42
11.2 Current Limitations 42

Chapter 12 Future Scope and Roadmap 44
12.1 Planned Improvements 44
12.2 Scaling Plan 44
12.3 Product Roadmap 45

Chapter 13 Product Launch Documentation 46
13.1 PRD Summary 46
13.2 Lean Canvas 46
13.3 Cost Estimate 47
13.4 Risks and Mitigation 48

13.5 KPIs After Launch 48
Chapter 14 Conclusion **49**
References **50**
Appendix A Questionnaire **55**
Appendix B Survey Data **56**
Appendix C Redis Keys **57**
Appendix D Code Excerpt **58**
Appendix E User Manual **59**

LIST OF FIGURES

2.1 Reported cyber financial fraud losses (2023 to 2024) and "digital arrest" cases (2022 to 2024) 3
3.1 Illustrative journey of Persona 1 through a fake KYC scam, without and with ScamShield (1 = distressed, 5 = at ease) 8
5.1 How a payment flows through ScamShield 12
8.1 System architecture 21
8.2 Level 0 data flow diagram 22
8.3 Level 1 data flow diagram 23
8.4 Use case diagram 24
8.5 Sequence diagram of checking and confirming a payment 25
8.6 Class diagram of the risk decision components 26
8.7 Entity relationship diagram (configuration tables not shown) 27
9.1 Project timeline 32
9.2 Wallet dashboard 33
9.3 A MEDIUM risk payment with reasons and the three questions 34
9.4 A HIGH risk payment, with "Cancel and report this account" as the main button 34
9.5 Analyst console with the live alert queue and alert detail 35
9.6 Scam simulator: a mule collection run moving from LOW to MEDIUM as more people pay the same account 35
9.7 Model and evaluation page 36
10.1 Precision-recall curves on the test period 39
10.2 Share of scams caught at the MEDIUM threshold, by scenario 39
10.3 Permutation feature importance (top 12 features) 40
13.1 Lean Canvas for ScamShield 47

LIST OF TABLES

2.1 Scale of payment fraud in India 3
3.1 User personas 6
3.2 User personas (continued) 7
4.1 TAM, SAM and SOM (yearly) 9
4.2 Feature comparison (✓ yes, ◐ partly, ✗ no, ? not public) 10
6.1 Functional requirements 14
6.2 Non-functional requirements and whether we met them 15
8.1 Technology choices 22
8.2 Main API endpoints 28
8.3 Default rules (all editable at runtime) 29
9.1 Development phases 32
9.2 Team roles 33
10.1 Main test cases (all passed) 37
10.2 Results on the test period at the MEDIUM threshold (score ≥ 35) 38
12.1 Product roadmap 45
13.1 Product requirements summary for a pilot release 46
13.2 Estimated first-year pilot budget (₹) 47
13.3 Main risks 48
13.4 KPIs and pilot targets 48
B.1 Coding sheet for survey responses 56
C.1 Redis keys used by the API and worker 57

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
APP fraud	Authorised Push Payment fraud
CERT-In	Indian Computer Emergency Response Team
CFCFRMS	Citizen Financial Cyber Fraud Reporting and Management System
DFD	Data Flow Diagram
DPDP Act	Digital Personal Data Protection Act, 2023
ER	Entity–Relationship
FN / FP / TN / TP	False Negative / False Positive / True Negative / True Positive
FPR	False-Positive Rate
FRI	Financial Fraud Risk Indicator (Department of Telecommunications)
FRM	Fraud Risk Management
GBM	Gradient-Boosting Machine
I4C	Indian Cyber Crime Coordination Centre
IST	Indian Standard Time
JWT	JSON Web Token
KPI	Key Performance Indicator
KYC	Know Your Customer
ML	Machine Learning
NCRP	National Cyber Crime Reporting Portal
NPCI	National Payments Corporation of India
OTP	One-Time Password
PR-AUC	Area Under the Precision–Recall Curve
PRD	Product Requirements Document
PSP / TPAP	Payment Service Provider / Third-Party Application Provider
RBAC	Role-Based Access Control
RBI	Reserve Bank of India
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SAM / SOM / TAM	Serviceable Available / Serviceable Obtainable / Total Addressable Market
SRS	Software Requirements Specification
SSE	Server-Sent Events
UAT	User Acceptance Testing
UPI	Unified Payments Interface

UVP	Unique Value Proposition

CHAPTER 1

Introduction

1.1 Background

UPI has become the default way to pay in India. In FY2025-26 it handled about 24,162 crore transactions worth ₹314.23 lakh crore [1, 2], and by June 2026 around 55.49 crore people had been onboarded [3]. NPCI publishes these numbers every month [4]. Paying a vegetable seller, a hostel mess or an electricity bill now takes a few seconds.

The same speed is what makes UPI useful to fraudsters. Government replies in Parliament put UPI fraud at about 13.42 lakh cases (₹1,087 crore) in FY2023-24 and 12.64 lakh cases (₹981 crore) in FY2024-25 [5]. Across all channels, reported cyber financial fraud losses were ₹22,845.73 crore in 2024, up from ₹7,465.18 crore in 2023 [6].

What surprised us when we started reading about this is that most of these cases are not hacks. The victim opens their own app, types the amount, enters their own PIN and sends the money, because someone on the phone has convinced them it is a KYC fee, a refund, a police deposit or a job registration charge. This is usually called authorised push payment (APP) fraud. PINs and OTPs cannot stop it, because the right person is entering them.

1.2 Motivation

Messages like "your KYC will expire today" or "your electricity will be disconnected tonight" are something almost every UPI user in India has seen, and surveys show that many people actually lose money to them: in a 2025 LocalCircles survey, about one in five UPI-using families said they had faced UPI fraud since 2022 [7]. Once the money is sent it moves through mule accounts quickly, and getting it back is slow. The government's fraud reporting system has saved more than ₹11,158 crore by June 2026 [8], which is a lot, but it is still small compared to what is lost.

So we asked a simple question: what if the app checked the payment before the money left, and warned the person in plain words when something looked wrong? The platform already knows things the payer does not, for example that the receiving account was created two days ago, or that fifteen strangers paid it today. Fraud detection research has used this kind of behavioural signal for a long time [9, 10]. The part we felt was missing was the payer side: telling the person why, at the moment they are about to pay.

The project also covered most of what we have studied: machine learning on very imbalanced data, backend design where a few hundred milliseconds matter, database consistency for money, and UI design that people actually pay attention to.

1.3 Objectives

The objectives we set at the start were:

1. Score every payment for scam risk before it is authorised, and return a 0 to 100 score, a band (LOW, MEDIUM or HIGH) and the reasons in plain language, within 300 ms at the 95th percentile.
2. Combine simple rules with a calibrated ML model, and check on a fair time-based test whether this hybrid catches more scams than rules alone at the same false alarm rate. We agreed to report the result even if the hybrid lost.
3. Respond in proportion to risk: nothing for LOW, a warning with a few questions for MEDIUM, and a forced pause with re-typing of the UPI ID for HIGH.
4. Give fraud analysts a live alert queue, and let their decisions block confirmed scam accounts for every user.
5. Keep payments working even when the ML service is slow or down.

1.4 Scope

We built a working UPI-style wallet with fake money, the full risk pipeline, the payer screens, an analyst console and an admin console, plus a synthetic data generator and an evaluation setup. We did not connect to any real bank or to NPCI, we did not handle card or net banking fraud, and we could not test on real fraud data, since students do not get access to it. All users, payments and scams in this report are synthetic.

1.5 Organisation of the Report

Chapter 2 describes the problem in more detail and Chapter 3 covers user research. Chapter 4 looks at the market and existing solutions, Chapter 5 gives our proposed solution and business model, and Chapter 6 lists the requirements. Chapter 7 covers legal and compliance issues. Chapters 8 and 9 explain the design and implementation, Chapter 10 the testing and results, and Chapters 11 to 13 the limitations, future plans and launch documents. Chapter 14 concludes.

CHAPTER 2

Problem Definition

2.1 Problem Statement

People using instant payment systems like UPI are being tricked into sending money to fraudsters themselves. Current safety checks confirm who is paying (PIN, OTP, device) but not whether the payment makes sense, and most help comes only after the money is gone. There is no common tool that, at the moment of payment, estimates how risky that particular payment is, explains the reason to the payer, and slows down only the risky ones without bothering everyone else.

2.2 Why It Is a Real Problem

Table 2.1 collects the main numbers we found. Figures from Parliament replies are cited through the news reports that published them.

Table 2.1: Scale of payment fraud in India

Indicator	Period	Value	Ref.
UPI fraud cases / amount	FY2023-24	13.42 lakh / ₹1,087 cr	[5]
UPI fraud cases / amount	FY2024-25	12.64 lakh / ₹981 cr	[5]
UPI fraud amount, part year (provisional)	FY2025-26 to Dec	₹805 cr	[11]
Cyber financial fraud losses, all channels	2023 / 2024	₹7,465 cr / ₹22,846 cr	[6]
"Digital arrest" cases / amount	2024	1,23,672 / ₹1,935.51 cr	[12]
Families reporting UPI fraud since 2022	2025 survey	about 1 in 5	[7]
Victims who never filed a complaint	2025 survey	about 51 %	[13]

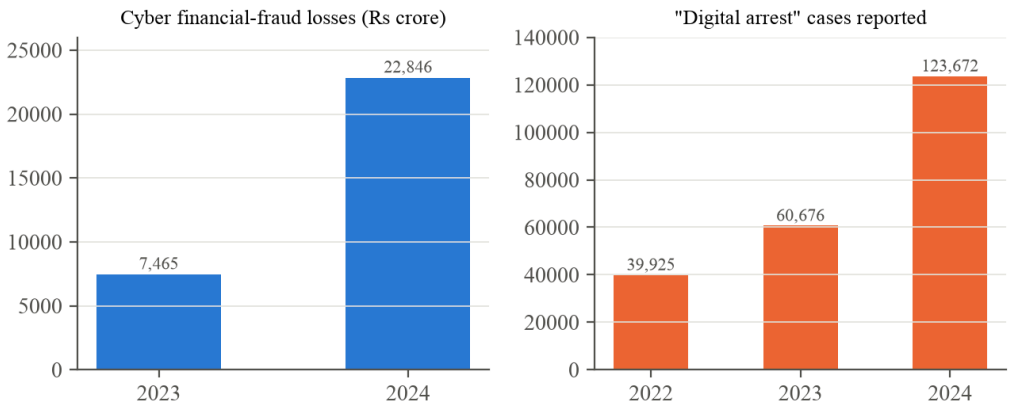


Figure 2.1: Reported cyber financial fraud losses (2023 to 2024) and "digital arrest" cases (2022 to 2024)

A few things stand out. Losses roughly tripled from 2023 to 2024. There are more than ten lakh UPI fraud cases a year, which is far too many to handle by hand. RBI's data for FY2024-25 shows card and internet frauds were about two-thirds of bank fraud cases but only about 7 % of the amount [14], which means a very large number of small losses to ordinary people, the kind that rarely get investigated in depth. And about half the victims in the LocalCircles survey never complained at all [13], so the real numbers are higher.

The common patterns are well known by now. In a fake KYC call, the victim is told their account will be blocked unless they pay a small "verification" amount. In a "digital arrest", a caller posing as police or customs keeps the victim on a video call for hours and demands a deposit. Job and task scams ask for a registration fee and then bigger "deposits", and the receiving account is a mule collecting from dozens of people a day. In a remote-access scam the victim installs a screen-sharing app "for support" and several quick payments of similar amounts follow. Account takeover uses stolen credentials from a new phone, often at night, to send most of the balance to someone new.

2.3 Root Causes

We think the problem comes from five things working together.

Psychology. Scams use authority, urgency and fear [15]. A person who has just been told they will be arrested is not going to calmly check the payee's details.

Authentication is not intent. The UPI PIN proves the account holder is present. It says nothing about whether they understand where the money is going.

The payer has less information than the platform. The system knows that an account is two days old or has been reported three times. The payer only sees a name.

Payments are instant and final. Money is passed through mule accounts within minutes, so recovery after the fact is hard.

Warnings are generic. Messages like "never share your OTP" are shown to everyone all the time, and studies show people learn to ignore warnings they see often [16, 17].

It has not been solved yet for practical reasons. Fraud data sits inside each bank and is sensitive, so it is hard to share or research. False alarms are expensive at UPI's volume: UPI averaged about 66 crore payments a day in FY2025-26, so even a 1 % false alarm rate would interrupt around 66 lakh genuine payments every day. Scoring has to finish in milliseconds. Black-box scores are hard to justify to customers, while rule-only systems miss new patterns. And scammers keep changing their scripts, so models trained on old data slowly go stale [18].

2.4 Who Is Affected

Senior citizens are the main target of "bank", "pension" and "digital arrest" calls and often lose savings. Students and young earners get pulled into job and task scams, usually in small repeated payments, and many are too embarrassed to report it. Small shopkeepers face fake refund and QR code tricks, and their normal pattern of receiving money from many people can look like a mule account to a careless system. Banks and payment apps carry the support load and the damage to their reputation, and in some countries they now have to refund victims [19]. The police deal with huge complaint volumes across states. Over time, all of this makes people trust digital payments less.

CHAPTER 3

User Research

We wanted the design decisions (which signals to use, how much friction is acceptable, how to word a warning) to come from evidence and not only from our own guesses. We used three sources: published government data and large surveys, especially the 2025 LocalCircles survey of over 32,000 respondents from 365 districts [7]; our own questionnaire (Appendix A); and research on how people react to security warnings [16, 20, 17].

3.1 User Personas

The four personas below are composites [21]. They do not describe real people, but each is based on typical users described in the sources above. The same four user types are used in our synthetic data generator (Section 8.8), so the system is designed and tested for the same people.

Table 3.1: User personas

	Kamala Rao, 67	**Rohan Kulkarni, 20**
Background	Retired teacher in Hubballi. Lives on a pension. Her son set up her UPI app. About 3 payments a day, ₹100 to ₹800, in daytime.	Engineering student in Dharwad. Monthly allowance. About 7 small payments a day (canteen, auto, recharge), often late at night.
Goals	Pay her own bills without troubling her son; keep her savings safe.	Quick payments with no delay; earn some money part-time.
Pain points	Trusts callers who sound official. Scared of her account being "blocked". Finds small text hard to read.	Gets irritated by any extra step. Sees "work from home" offers on Instagram. Would not tell his parents if he lost money.
Behaviour	Follows instructions step by step and panics under time pressure.	Confident with apps, ignores generic warnings, often pays new people.
Main risk	Fake KYC or bank calls, "digital arrest".	Job and task scams paying a mule account.

Table 3.2: User personas (continued)

	Imran Shaikh, 38	**Priya Nair, 29**
Background	Runs a kirana shop in Belagavi. Receives about 25 payments a day and pays suppliers in batches of 3 to 5 within minutes.	Fraud operations analyst at a mid-size bank. Reviews flagged transactions from a queue. (This is our B2B user.)
Goals	Keep the counter moving, pay suppliers on time, avoid "I sent you extra, please refund" tricks.	Look at the riskiest cases first and make decisions she can defend.
Pain points	Cannot afford false alarms on routine supplier payments. Uses a second phone at the counter.	Too many alerts, scores with no reasons, and no easy way to see that ten people paid the same new account today.
Behaviour	High volume and repetitive; his normal pattern looks like "velocity" and "new device".	Wants all the context on one screen and a quick way to act.
Main risk	Fake refunds, QR swaps, account takeover.	Not a victim; she needs better tools.

3.2 Questionnaire

We prepared a 20-question online questionnaire that takes under five minutes on a phone. It is given in full in Appendix A. It has five sections: about the respondent and how often they use UPI (Q1 to Q5); whether they have faced scam attempts and what kind (Q6 to Q10); losses and whether money was recovered or reported (Q11 to Q13); how they feel about warnings, a 10-second pause and three quick questions before a risky payment (Q14 to Q18); and whether they are comfortable with payment patterns being analysed for fraud prevention (Q19 to Q20). Our hypotheses were that most people have faced scam attempts, that victims rarely get money back or report it, and that people would accept a short delay on a risky payment if it is rare and explained.

3.3 Responses and Key Insights

The published surveys already support the first two hypotheses. About 20 % of UPI-using families in the LocalCircles survey had faced fraud since 2022, around 40 % of victims lost money after clicking a payment link [7], and about 51 % never complained [13]. "Digital arrest" cases, which are purely social engineering, went from 39,925 in 2022 to 1,23,672 in 2024 [12].

TEAM TO COMPLETE — Add the results of your own questionnaire here: number of respondents, when and how it was shared, and the percentage for each key question (Q6, Q8, Q12, Q15, Q17, Q19). Appendix B has a coding sheet for this. Only report data you actually collected.

From the reading and the personas, these are the points that most changed our design:

- Since the real owner enters the PIN, we score the payment itself and not just the login, and we ask three questions about things the data cannot see, like whether someone is on the phone pushing them to pay.
- People ignore warnings they see often, so LOW payments show nothing at all, and warnings name specific facts ("this account is 2 days old") with at most three reasons.
- Scammers rely on hurry, so a HIGH payment gets a 10-second pause and the UPI ID must be typed again. "Cancel and report" is the biggest button.
- A victim being coached on the phone may lie, so the answers to the questions can only increase the risk score, never reduce it.
- Shopkeepers and festival shoppers behave unusually but honestly, so each user is compared with their own 30-day history, and our training data includes these cases.
- Analysts like Priya need reasons, the ML score and the payee's history together, and one click to block a confirmed scam account for everyone.

3.4 User Journey Map

Figure 3.1 follows Kamala through a fake KYC call, today and with ScamShield. The emotion scores are illustrative. Today there is nothing between the phone call and the PIN. With ScamShield, the recipient card already shows "first payment to this person" and "on ScamShield for 2 days". The check returns MEDIUM, and when she answers that someone called asking her to pay urgently, it moves to HIGH. She gets a pause, sees the reasons, and cancels and reports the account, which raises its risk for everyone else.

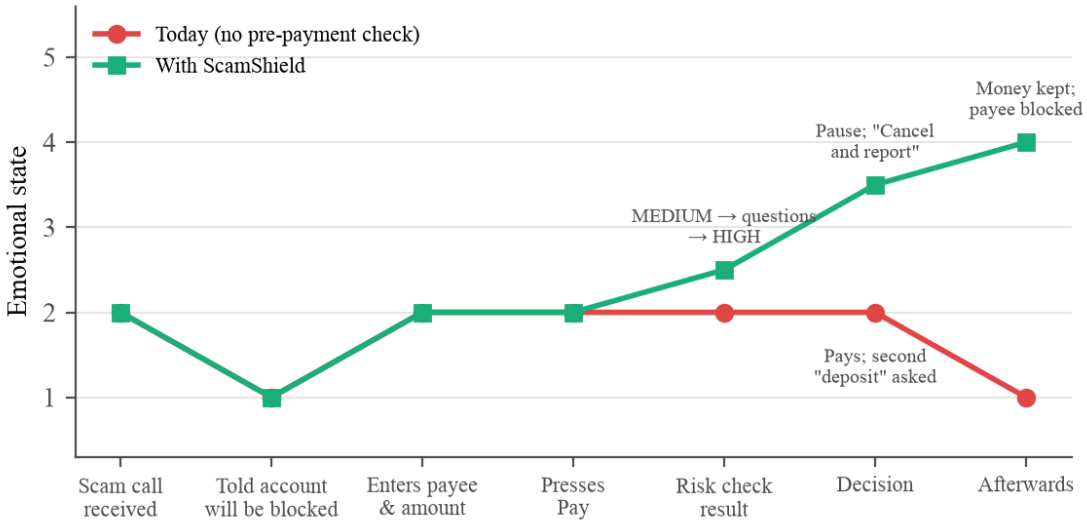


Figure 3.1: Illustrative journey of Persona 1 through a fake KYC scam, without and with ScamShield (1 = distressed, 5 = at ease)

CHAPTER 4

Market and Competitive Analysis

4.1 Market Size

We see ScamShield as a B2B product. The customers would be institutions that send UPI payments for their users, like cooperative banks, small finance banks and smaller payment apps (TPAPs), and the warning screens would be given to them as an SDK. End users would not pay anything.

For the total market we took published estimates of the Indian fraud detection and prevention market. These vary a lot: MarketsandMarkets puts it at about US$ 990 million in 2026, growing 22.4 % a year [22], while IMARC says US$ 1.4 billion in 2024, growing about 20 % a year [23]. We used the lower one. For the serviceable market we worked from UPI volume: about 241.6 billion transactions in FY2025-26 [1]. If each payment check cost 2 paise (our own assumption), checking every UPI payment would be worth about ₹483 crore a year. For what we could realistically get in five years, we assumed 1 % of UPI volume from 5 to 10 smaller banks and apps.

Table 4.1: TAM, SAM and SOM (yearly)

Level	What it covers	Estimate	How we got it
TAM	All fraud detection and prevention spending in India	≈ ₹8,400 crore	US$ 990 million [22] at ₹85 per US$
SAM	Pre-payment scam checks on UPI payments	≈ ₹483 crore	241.6 billion payments × ₹0.02
SOM	Our realistic share within 5 years	≈ ₹4.8 crore	1 % of UPI volume × ₹0.02

The SOM is small on purpose. Big banks and the large UPI apps build this in-house or buy from established vendors. Our first customers would be smaller players without a data science team. Regulation also helps here: RBI's 2025 authentication directions expect risk-based checks on top of two-factor authentication from April 2026 [24], and RBI's own MuleHunter.AI shows that banks are willing to use AI for fraud [25, 26].

4.2 Existing Solutions

Banks already run fraud risk management platforms such as FICO Falcon, Clari5, SAS or Feedzai. These monitor transactions in real time, but they are expensive, aimed at large banks, and usually hold or decline a payment quietly without telling the payer why. MuleHunter.AI, built by RBI Innovation Hub and used by about two dozen banks, finds mule accounts on the receiving side [25, 26]. The Department of Telecommunications' Financial Fraud Risk

Indicator rates mobile numbers by fraud risk, and some payment apps use it to block payments [27]. Sanchar Saathi and Chakshu let citizens report fraud calls, which has led to lakhs of numbers being disconnected [28]. The 1930 helpline and CFCFRMS help freeze money after a complaint [8]. The big UPI apps show some warnings of their own, but how these work is not public. Caller ID apps like Truecaller flag spam calls but never see the payment. Abroad, the UK has made banks reimburse APP fraud victims since October 2024, with the cost split between the sending and receiving banks [19], which gives banks a strong reason to stop scams before payment.

4.3 Feature Comparison

Table 4.2: Feature comparison (✓ yes, ◐ partly, ✗ no, ? not public)

Feature	Scam-Shield	Bank FRM	Mule-Hunter	DoT FRI	App warnings	1930
Acts before money moves	✓	✓	◐	✓	✓	✗
Explains the risk to the payer	✓	✗	✗	◐	◐	✗
Friction depends on risk level	✓	◐	✗	◐	?	✗
Asks the payer about context	✓	✗	✗	✗	?	✗
Signal across many payers (mule)	✓	◐	✓	◐	?	◐
Analyst console and feedback	✓	✓	?	✗	✗	◐
Affordable for small institutions	✓	✗	◐	✓	✗	✓
Proven on real fraud data	✗	✓	✓	✓	?	n/a

The last row is the important one. Existing systems have been tested on real fraud, and ours has not. Where we are different is the payer side of the problem.

4.4 SWOT Analysis

Strengths

- Rules and calibrated ML together, with clear reasons
- Warnings and friction aimed at the payer
- Fast (p95 about 110 ms) and keeps working if ML is down
- Evaluation that anyone can rerun

Weaknesses

- Only synthetic data so far
- No real bank or NPCI integration
- The questions depend on the user being honest
- Small student team with no sales experience

Opportunities

- RBI's push for risk-based checks [24]
- Market growing about 20 % a year
- Many small banks with no data science team
- Public signals like DoT FRI to add later

Threats

- Big apps and vendors adding similar features
- Scammers coaching victims through warnings
- Cost of compliance and audits
- False alarms losing user trust

4.5 Unique Value Proposition

Our pitch in one line is: *stop the scam before the money moves.* ScamShield does not just produce a score for the bank. It tells the payer in one sentence why a payment looks wrong, asks the questions an analyst would ask, and only slows them down when the risk is real. When an analyst confirms a scam account, every other user is protected from it. And because it is light and runs on modest hardware, smaller banks and apps can afford it.

CHAPTER 5

Proposed Solution

5.1 Overview

ScamShield puts a risk check between "the user presses Pay" and "the money moves". A payment happens in two steps. The first step, *check*, works out the risk and never moves any money. The second step, *confirm*, moves the money, but only if the app sends back a short-lived token from the check and has completed whatever the risk band requires. Figure 5.1 shows the flow.

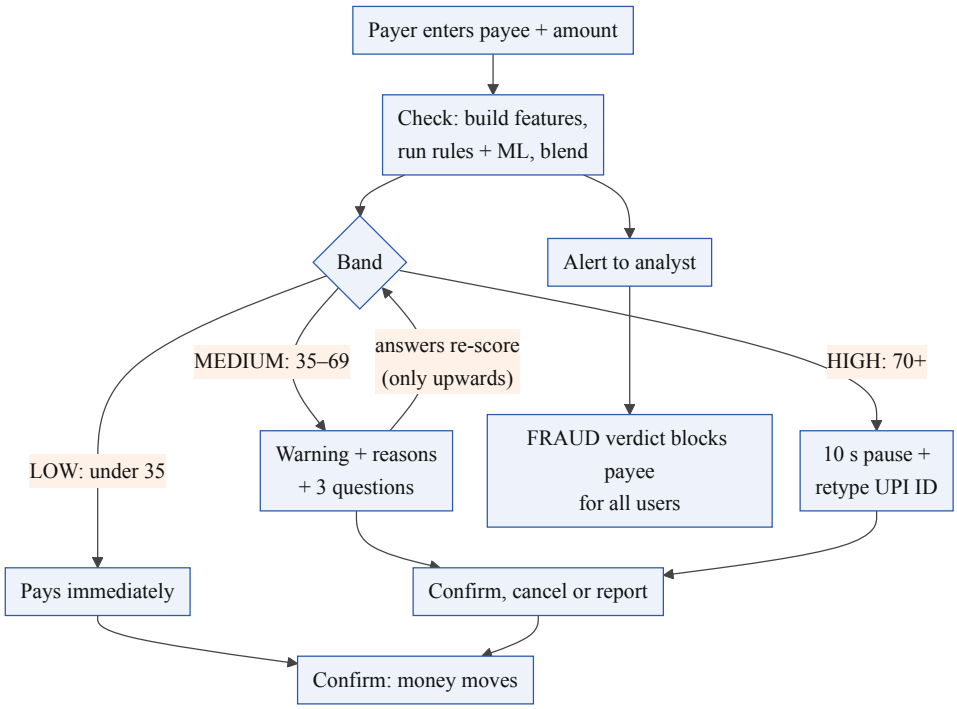


Figure 5.1: How a payment flows through ScamShield

This addresses the root causes from Section 2.3 directly. The payment is scored on behaviour even when the PIN is correct. The payer is shown what the platform knows, such as the account's age and reports. The pause and re-typing break the hurry that scammers depend on. The three questions catch the phone call that the data cannot see. LOW payments get no warning at all, so warnings stay rare. And because reports and analyst verdicts apply to every user, one victim's report protects the next person.

5.2 Key Features

The prototype already has these features:

- A hybrid risk engine with 20 features, 11 rules and a calibrated gradient boosting model, blended 40:60 into a score from 0 to 100.
- Two hard overrides: payments to an analyst-confirmed scam account are blocked, and "someone asked me for an OTP or to install an app" always means HIGH.
- Reasons in plain English for the user (top three) and full detail for analysts.
- The LOW, MEDIUM and HIGH screens described above, with the three scam questions.
- A 30-day baseline per user, and a reputation score per payee built from reports, account age and how many different people paid it in the last day.
- An analyst console with live alerts, and an admin console with a rules editor, model evaluation, retraining, a scam simulator and a system health page.
- A fallback to rules only if the ML service is slow or down, and payments that cannot be duplicated or double spent.

Later we would like to add graph-based detection of mule networks, external signals like DoT FRI, warnings in Hindi and Kannada, a "trusted contact" confirmation for senior citizens, and safer ways to roll out new models.

5.3 Business Model

We would charge institutions, not users. The main income would be a fee per checked payment (we assumed ₹0.02, dropping to ₹0.01 above 10 crore checks a month), plus a monthly subscription for the analyst and admin consoles (₹50,000 to ₹2,00,000 depending on seats), a one-time onboarding and tuning fee (₹5 to 15 lakh), and an on-premise licence for institutions that cannot use a hosted service. For the customer, the value is fewer fraud losses and support calls, less damage to their reputation, and something to show the regulator for risk-based checks.

5.4 Go-to-Market Strategy

We would start with a pilot at one cooperative or small finance bank, or a campus payment app. For the first few months ScamShield would run in shadow mode, scoring real payments without showing anything to users, so we can measure how it actually performs. Then we would switch on the warnings for a group of users who agree to it. With those results as a case study, the next step is other cooperative banks (through their federations and technology providers) and smaller payment apps, with a free tier up to some monthly volume. After that we would apply to RBI's regulatory sandbox and Innovation Hub programmes and try to partner with core banking and UPI switch vendors so ScamShield can be added as a plug-in.

CHAPTER 6

Requirements Specification

We wrote the requirements loosely following IEEE Std 830 [29]. There are three kinds of users. A **user** holds a wallet and sends money. An **analyst** can do everything a user can, and also reviews alerts. An **admin** can do everything an analyst can, and also manages rules, thresholds, the model, the simulator and system health.

6.1 Functional Requirements

Table 6.1: Functional requirements

ID	The system shall…
FR-1	let a person register, log in and out, get a UPI ID and ₹25,000 of demo money, and count failed logins as a risk signal.
FR-2	show balance, 30-day spending, transaction history with risk band, and warnings received.
FR-3	show the payee's name, age on the platform, report count and whether the user has paid them before.
FR-4	on a payment request, compute 20 features, run the enabled rules, get an ML probability, blend them into a 0 to 100 score and assign a band, without moving money.
FR-5	apply the two hard overrides (confirmed scam payee, OTP or app request).
FR-6	return reason codes, a one-line summary and the top three reasons in plain language.
FR-7	treat repeated check requests with the same Idempotency-Key as one request.
FR-8	issue a signed check token valid for 10 minutes, and require it to confirm.
FR-9	let LOW payments go through directly; show warning, reasons and three questions for MEDIUM and re-score on the answers (only upwards); require a 10-second pause and the exact UPI ID for HIGH.
FR-10	on confirm, debit, write ledger entries and complete the payment in one database transaction, with no double spending.
FR-11	let the user cancel and optionally report the payee, once per user per payee.
FR-12	create an alert for MEDIUM and HIGH decisions and show it to analysts live.
FR-13	show analysts the features, reasons, ML contributions and payee history, and accept a FRAUD or SAFE verdict; FRAUD marks the payee as confirmed fraud.
FR-14	let admins edit rules, weights and thresholds at runtime with a what-if preview, retrain the model, run scam scenarios and view system health.
FR-15	keep an audit log of verdicts, rule changes, retraining and simulator runs.

6.2 Non-Functional Requirements

Table 6.2: Non-functional requirements and whether we met them

Type	Requirement	Status
Performance	p95 of the check endpoint under 300 ms; ML call capped at 150 ms	Met, about 110 ms at 20 req/s
Availability	If ML fails, payments are still scored by rules and marked degraded	Met and tested
Consistency	No negative balance; only one of several parallel confirms succeeds	Met, tested with 5 at once
Security	bcrypt passwords, server-side role checks, input validation, rate limits, signed tokens	Met
Privacy	Device IDs stored only as hashes; no card or bank data	Met
Scalability	API and ML services stateless so they can be replicated	Designed, not load tested beyond one instance
Usability	No extra step for LOW; at most three reasons; mobile layout and dark mode	Met
Reproducibility	Same seed gives the same data, model and results	Met (seed 42)

6.3 Hardware and Software Requirements

For development and the demo, a machine with a 4-core 64-bit CPU, 8 GB RAM and about 10 GB of free disk is enough. We used an Apple silicon laptop. The client can be any modern browser, desktop or mobile. A pilot would need around two 4 vCPU / 16 GB virtual machines plus managed PostgreSQL and Redis.

On the software side, the stack runs on Docker with Compose v2 on macOS, Linux or Windows (WSL2). It uses Node.js 20 or later with TypeScript, Next.js 15 and React 19 for the web app, Express with Prisma for the API, Python with FastAPI and scikit-learn 1.6 for the ML service, PostgreSQL 16 and Redis 7. Testing uses Vitest, pytest, Playwright and autocannon.

6.4 Constraints and Assumptions

The main constraints were that we had no access to a real payment network or real fraud data, that the core build had to be done in about one week, that scoring had to be fast enough for a live payment, and that the whole thing had to run on one laptop for the demo. A real deployment would also have to meet RBI, NPCI and DPDP rules (Chapter 7).

We assumed that all time-of-day features use IST, that the maximum payment is ₹1,00,000, and that the device sends a stable random ID which the server only stores as a hash. "Payee age" means days since the payee was first seen on our platform. Failed attempts include both

wrong passwords and wrong UPI IDs at the re-type step. We also assumed that answers to the questions might not be truthful, and that only an analyst's verdict, not user reports alone, can block a payee.

CHAPTER 7

Policies, Legal and Compliance

The prototype only uses synthetic data, so the demo itself does not raise legal issues. A real launch would process personal and financial data of real people and make decisions about their payments, so this chapter lists what that would need. This is our reading of the rules as students, not legal advice. A real launch would need a lawyer and the partner bank's compliance team to review it.

7.1 Privacy and Data Protection

The main law is the **Digital Personal Data Protection Act, 2023** [30]. It allows personal data to be processed only for a lawful purpose, based on consent after a clear notice or on certain legitimate uses. The organisation holding the data (the Data Fiduciary) must limit use to that purpose, keep the data accurate, protect it with reasonable security, report breaches to the Data Protection Board and to the affected people, delete data when it is no longer needed, and provide a way to raise grievances. Failing to protect data can be penalised up to ₹250 crore. The **DPDP Rules, 2025** were notified in November 2025, and most of the main duties apply about 18 months later, around May 2027 [31], so anything we launch would have to comply from the first day.

Until then, Section 43A of the **IT Act, 2000** and the SPDI Rules, 2011 still apply to sensitive personal data, which includes financial information [32, 33]. Section 72A covers disclosing personal data in breach of contract. Sections 66C and 66D (identity theft and cheating by personation) describe the very offences we are trying to prevent. The **CERT-In directions of April 2022** require reporting cyber incidents within 6 hours and keeping logs for 180 days in India [34]. RBI's 2018 circular on payment data storage requires payment data to be stored only in India [35]. If the product were ever offered in Europe, the GDPR would also apply, including Article 22 on decisions made only by automated means [36].

Our draft privacy policy would say the following. We collect account details, payment details (amount, payee, time), a hashed device ID, login results, answers to the scam questions and reports filed. We do not collect contacts, SMS, call logs, location, card numbers or bank passwords. The data is used only to prevent fraud, run the wallet and meet legal duties, and never for advertising, credit scoring or sale. Features are kept for 12 months for retraining and disputes, and baselines for 30 days. Logs are kept for as long as the law requires, and after that data is deleted or anonymised. Data is shared only with the user's bank or payment app,

with law enforcement when required, and (for confirmed scam payees only) with industry fraud registries. Users can see a summary of their data, ask for corrections or deletion, raise a grievance, and see all their warnings and reports on the My Safety page.

7.2 Terms of Service and Consent

The terms would make clear that ScamShield gives warnings. It reduces risk, but it cannot catch every scam, and not every flagged payment is a scam. The user can still continue a MEDIUM payment after reading the warning, and a HIGH payment after the pause and re-typing. Only payments to analyst-confirmed scam accounts are fully blocked, and the account owner can appeal through the grievance channel. Answering the questions is voluntary. Reports must be made in good faith, and a report alone never blocks anyone. Consent would be taken through a separate, simple notice, not hidden in the wallet terms, and withdrawing it would be as easy as giving it. Because fraud checks are part of the payment service, we would also explain up front that withdrawing consent may limit payments through the partner app.

7.3 Security Policies

In the prototype, passwords are hashed with bcrypt (cost 10). Sessions use a signed JWT in an httpOnly, SameSite cookie [37], and login errors are generic. Every route checks roles on the server. All inputs are validated with zod, request bodies are limited to 100 kB, security headers are set with helmet, CORS is limited to the web app, and Redis rate limits apply to logins (20 per minute per IP) and payment checks (120 per minute per user). The check token is signed and tied to the user, transaction, amount and payee. Device IDs are hashed, and secrets live in an environment file that is not in the repository. Analyst verdicts, rule changes, retraining and simulator runs are written to an audit log.

Before production we would still need TLS everywhere, encryption at rest, a secrets manager and key rotation, hosting in an Indian region, multi-factor login for analysts and admins, two-person approval for rule changes, tamper-proof logs kept for 180 days, a written incident response plan (CERT-In within 6 hours, plus DPDP notifications), an external penetration test, and a check against OWASP ASVS Level 2 [38].

7.4 Accessibility and Ethics

Many of our likely users are older or new to digital payments. We aimed for WCAG 2.2 AA [39], in line with the Rights of Persons with Disabilities Act, 2016 [40]. Risk is never shown only by colour (each band also has an icon and a label), sentences are short, buttons are big, and the screens work at phone width and in dark mode. Screen reader testing and regional languages are still to be done.

On ethics, the biggest worry is false alarms on honest people, so we chose thresholds by looking at measured false alarm rates and LOW payments see nothing. We use no demographic features. Each user is only compared with their own history, though in a real deployment false alarm rates should be checked separately for groups like seniors and new users. The friction is used only to slow down a likely scam and never for any commercial purpose. Every decision has a readable reason, and a human analyst has to confirm before a payee is blocked. We have also tried to be open about the weak points of our results, which are listed in Chapter 11.

7.5 Payments Sector Rules

A bank or payment app using ScamShield would have to fit it into several RBI and NPCI requirements. RBI's 2021 Master Direction on Digital Payment Security Controls asks regulated entities for fraud risk controls including transaction monitoring [41], and ScamShield is one such control. RBI's 2024 Master Directions on Fraud Risk Management require early warning signals and reporting [42], and our alerts and audit log could feed into that. The 2025 Authentication Directions ask for risk-based checks from April 2026 [24]. NPCI's 2025 guidelines limit API calls [43], so scoring has to run inside the bank's own systems. RBI's outsourcing rules would apply if a bank used us as a vendor, which means audit rights, data location and business continuity plans. DoT's FRI could be used as an extra signal, subject to its access terms [27].

CHAPTER 8

System Design and Technical Details

8.1 System Architecture

ScamShield runs as six Docker containers started together with Docker Compose [44] (Figure 8.1). The **web** container is the Next.js front end. It also forwards `/api` calls to the API, so the login cookie stays on the same site. The **api** container is a stateless Express server that handles login, payments, alerts and admin functions. The **worker** uses the same code but runs background jobs. The **ml** container is a FastAPI service that trains the model on first start and then answers scoring requests. **PostgreSQL** is the main database and **Redis** holds fast-changing state like counters, caches, rate limits and job queues.

The main design idea was to keep the "hot path", everything needed before the payer sees a result, as short as possible. Anything that does not change the current decision (creating alerts, refreshing baselines, updating payee reputation, metrics, retraining) goes to a queue and is done by the worker.

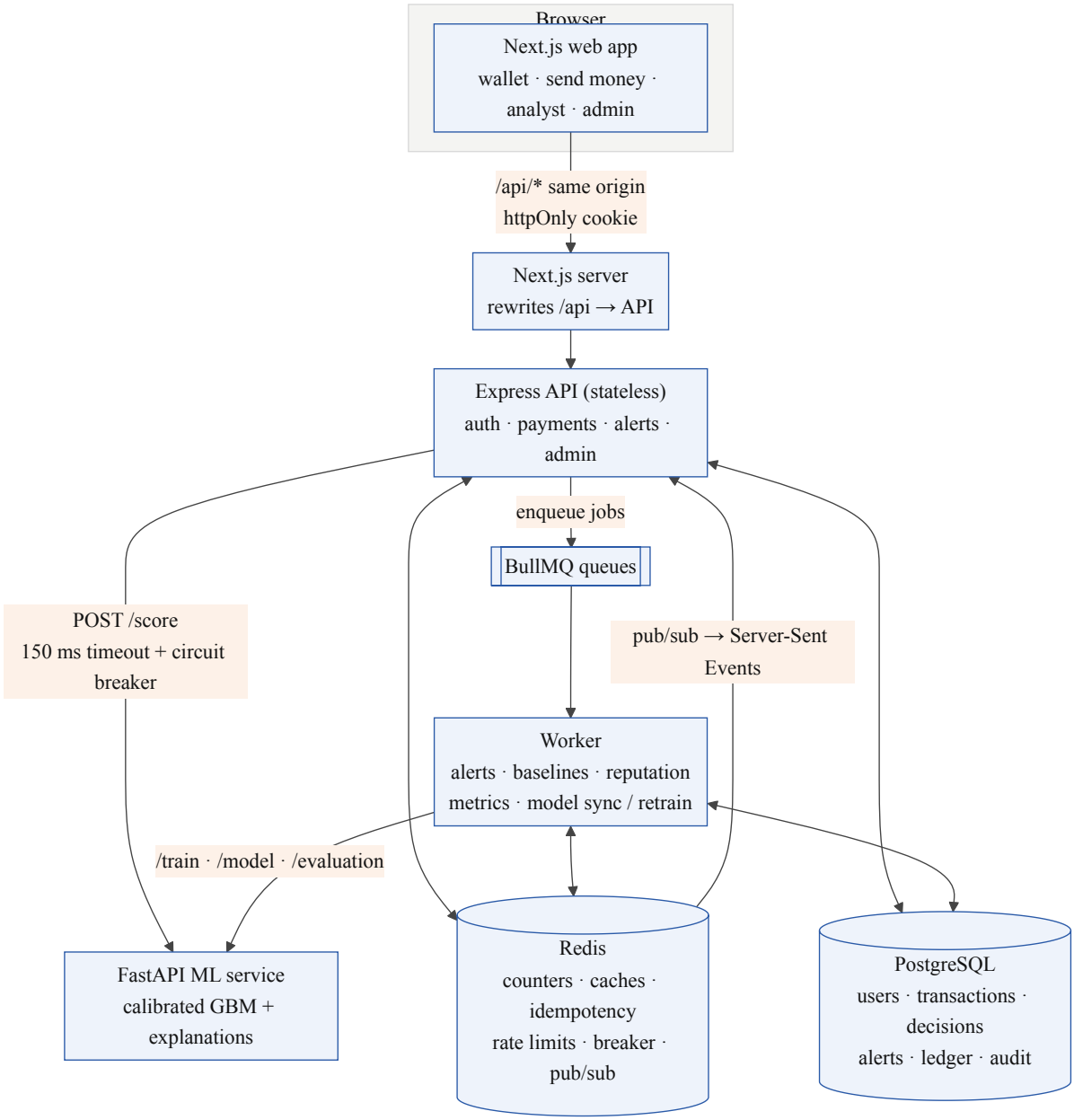


Figure 8.1: System architecture

8.2 Technology Stack

Table 8.1: Technology choices

Part	What we used	Why
Front end	Next.js 15, React 19, Tailwind, Radix UI, Recharts [45]	Server-side forwarding keeps cookies same-origin; Radix gives accessible modal dialogs; Recharts for the evaluation charts.
API	Node.js, Express, TypeScript, zod [46]	The rules and features are shared TypeScript code used by both web and API. zod validates input and gives us types.
Database	PostgreSQL 16 with Prisma [47, 48]	Money needs proper transactions and conditional updates. Prisma handles the schema and migrations.
Fast state	Redis 7 [49]	Sorted sets make sliding-window counts easy; atomic SET NX for idempotency; pub/sub for live alerts.
Queues	BullMQ [50]	Retries with back-off, and it runs on the Redis we already had. Kafka would be too much for this size.
ML	Python, FastAPI, scikit-learn [51, 52]	HistGradientBoosting is quick to train, needs no GPU and handles raw features. We skipped XGBoost, LightGBM and SHAP [53, 54, 55] to keep the container small.
Testing	Vitest, supertest, pytest, Playwright [56], autocannon	Integration tests on real PostgreSQL and Redis, browser tests and a load test.

8.3 Data Flow Diagrams

The Level 0 diagram (Figure 8.2) shows ScamShield as one process with its three kinds of users. The Level 1 diagram (Figure 8.3) breaks it into six processes: authenticate, build features, score risk, intervene and settle the payment, raise alerts and update reputation, and review and learn.

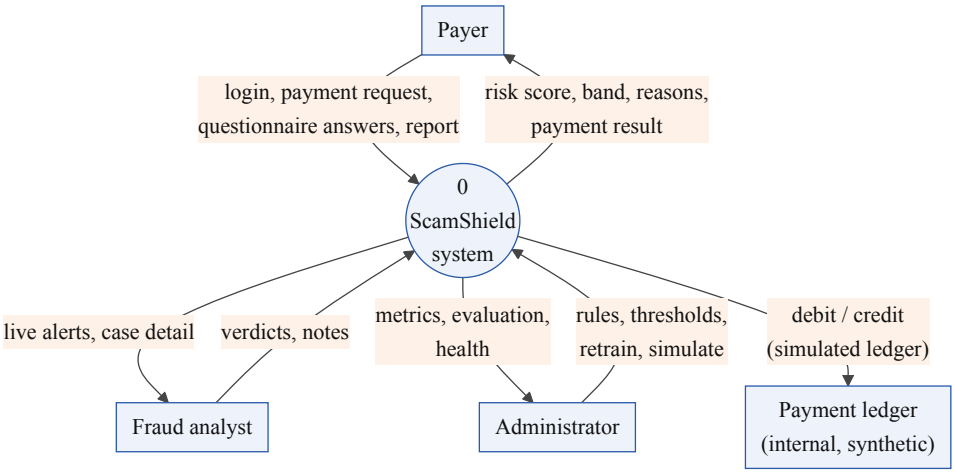


Figure 8.2: Level 0 data flow diagram

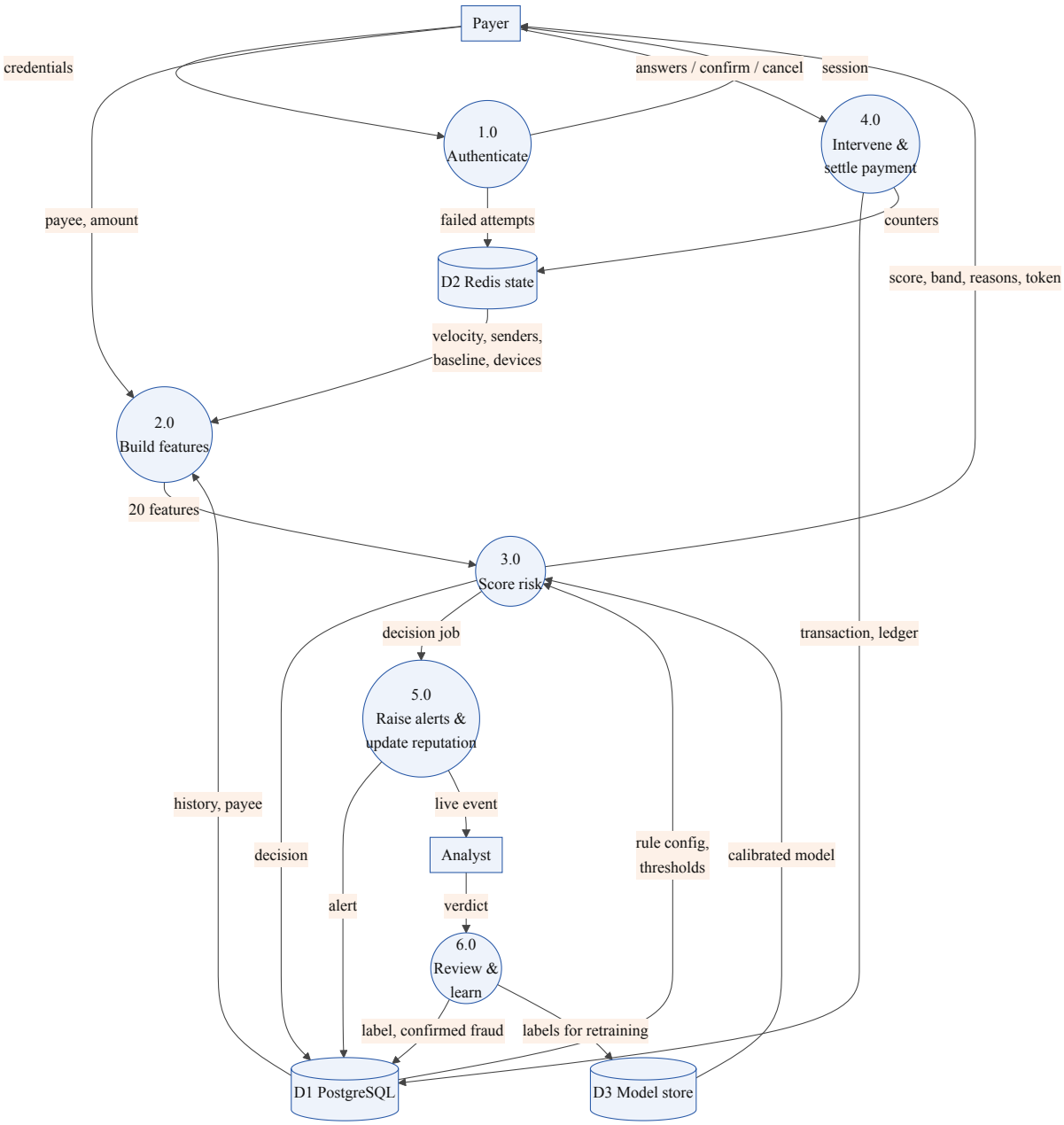


Figure 8.3: Level 1 data flow diagram

8.4 UML Diagrams

Figure 8.4 shows the use cases for each role. Figure 8.5 shows the order of calls when a payment is checked and confirmed. Figure 8.6 shows the main classes involved in making a decision. A transaction starts as PENDING (or BLOCKED if the payee is a confirmed scam account) and ends as COMPLETED or CANCELLED.

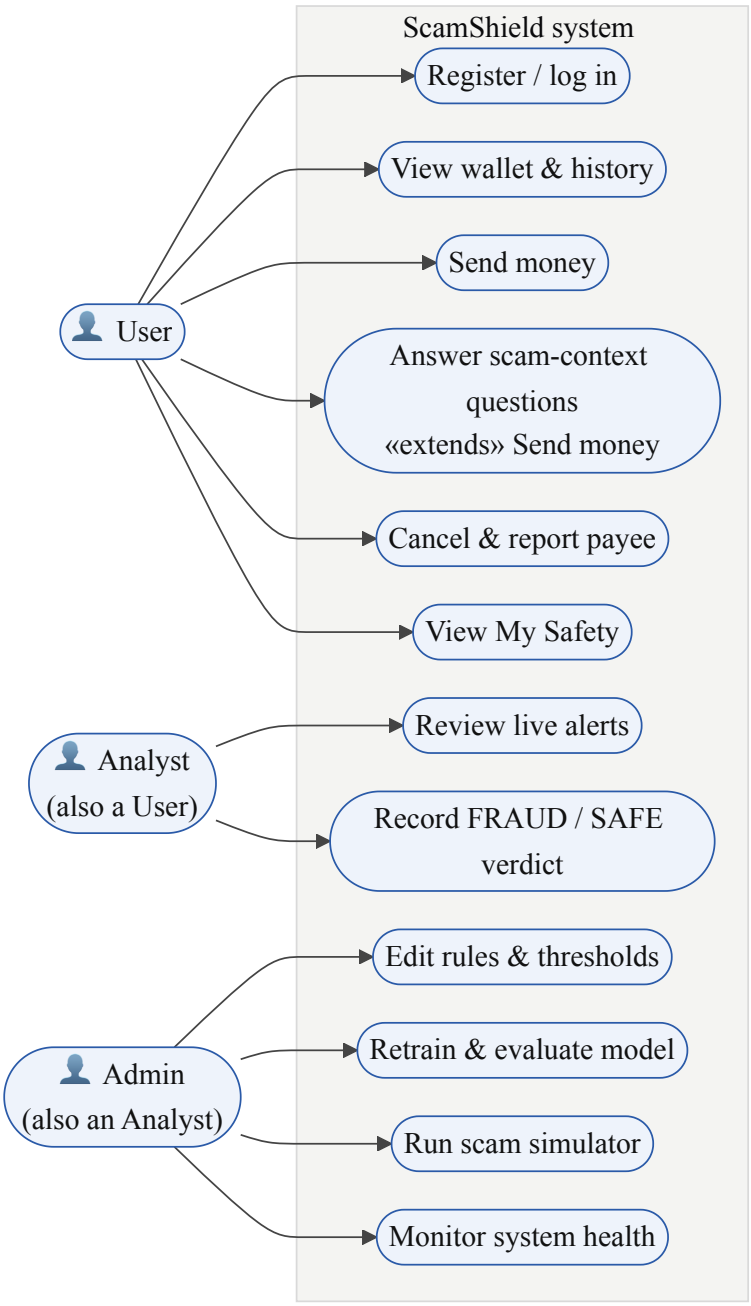


Figure 8.4: Use case diagram

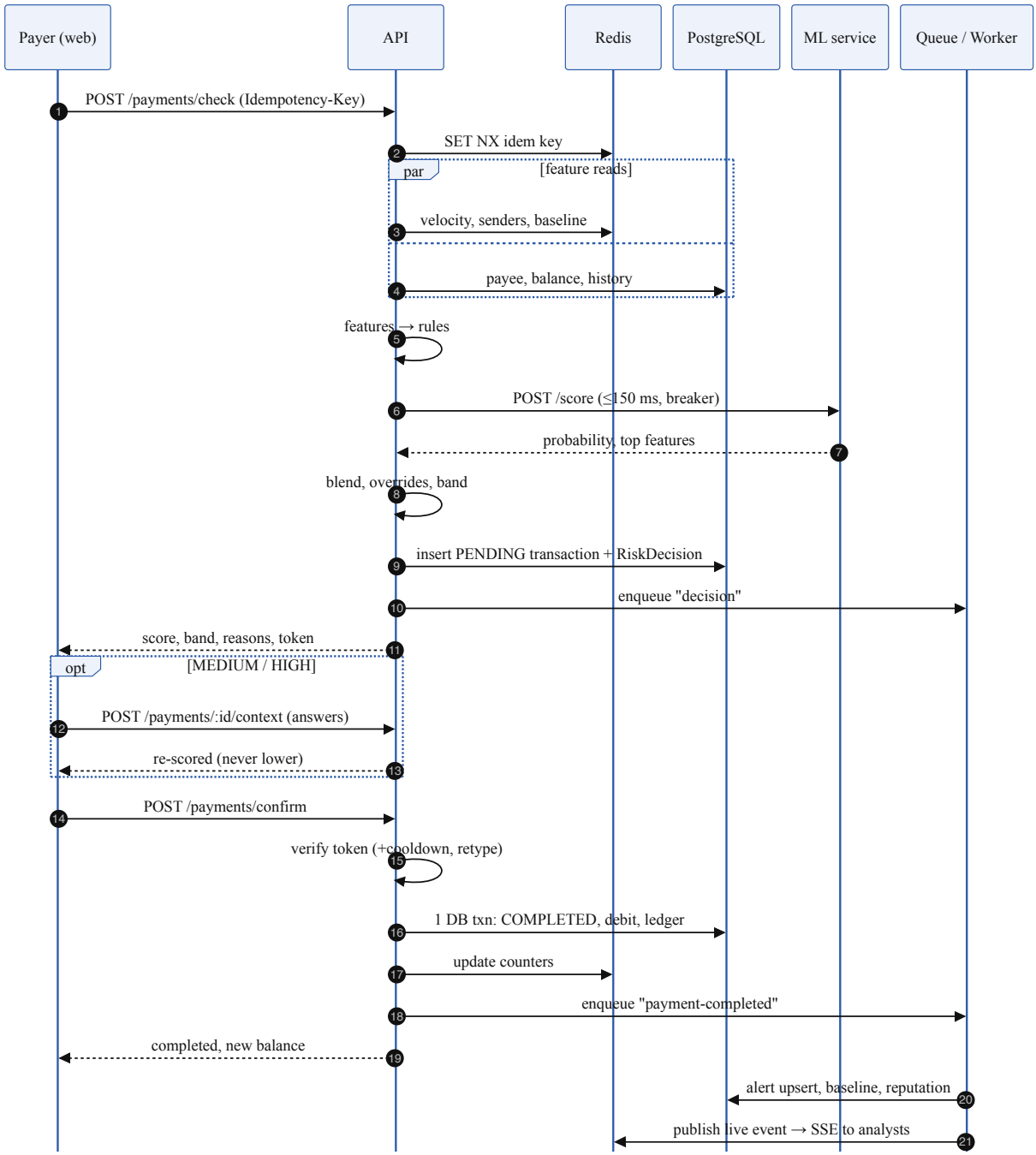


Figure 8.5: Sequence diagram of checking and confirming a payment

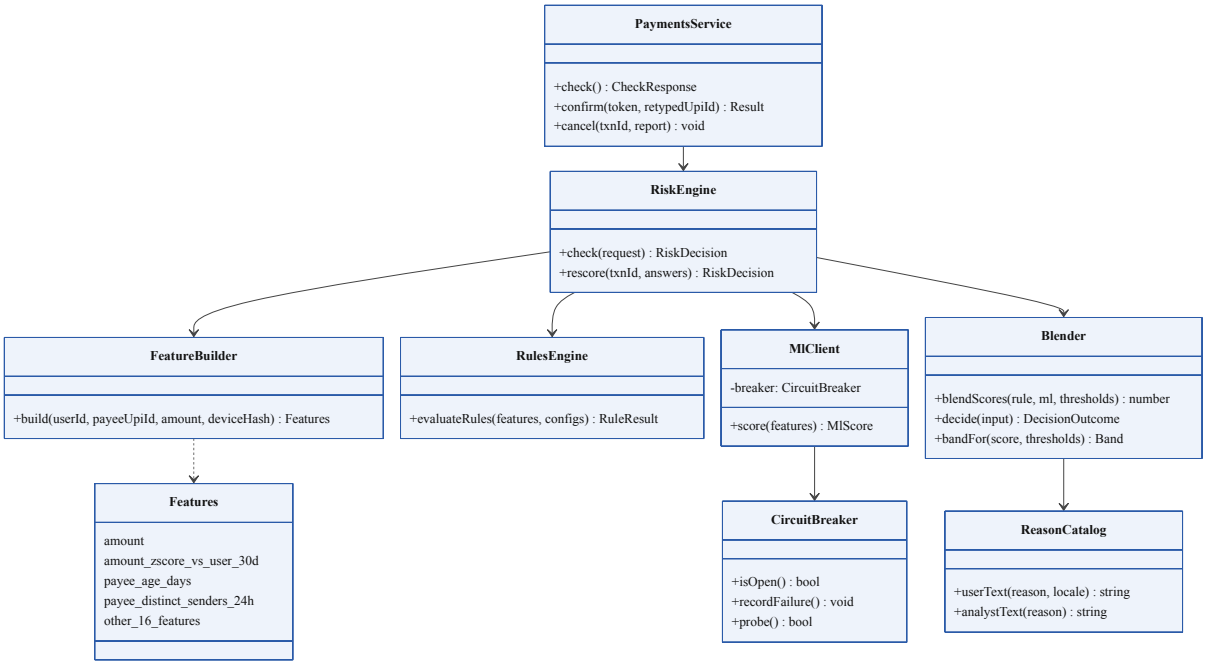


Figure 8.6: Class diagram of the risk decision components

8.5 Database Schema

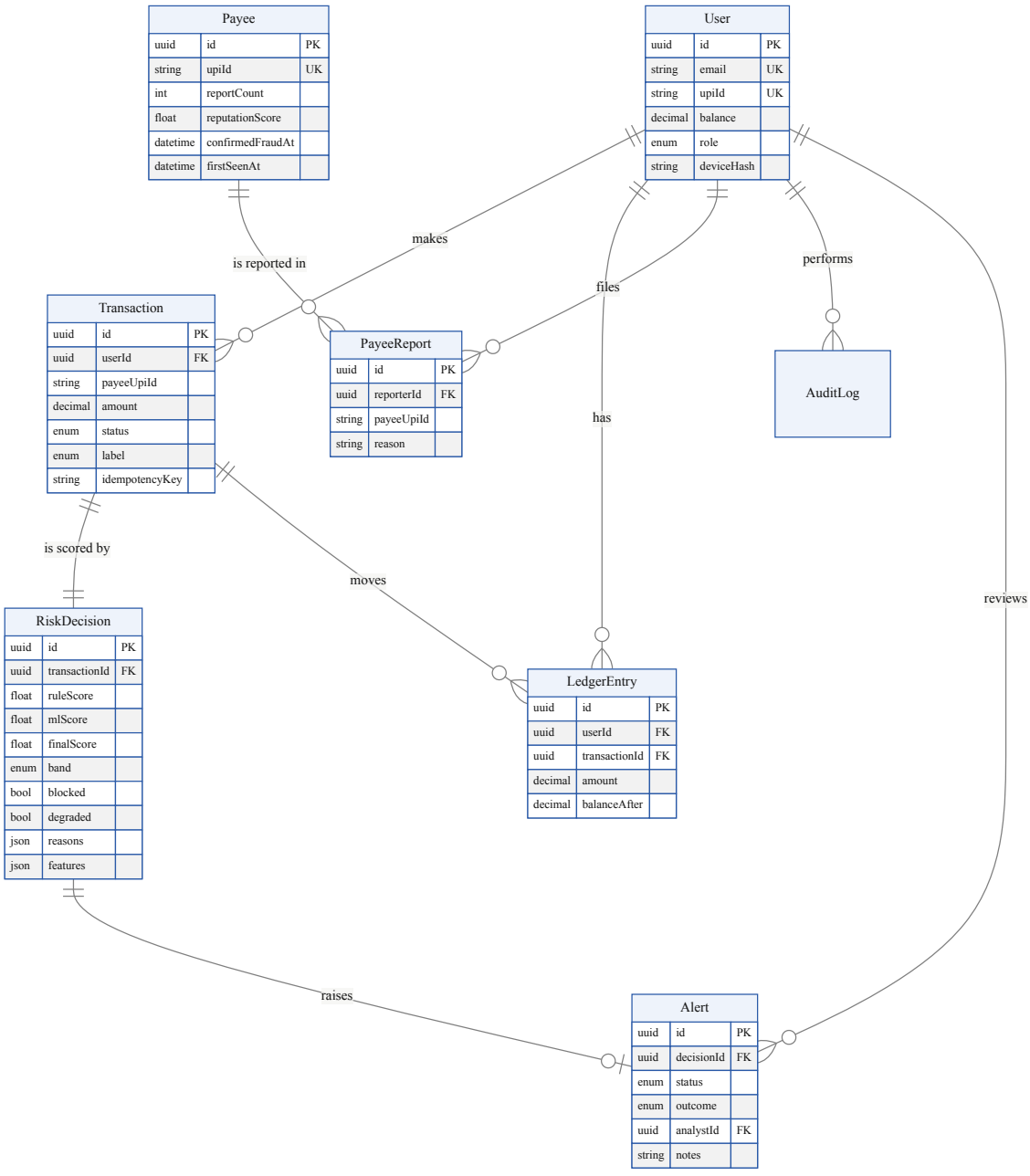


Figure 8.7: Entity relationship diagram (configuration tables not shown)

The main tables are **User** (role, UPI ID, balance, hashed registered device), **Payee** (every UPI ID ever paid, with first-seen date, report count, reputation and a confirmed-fraud date), **Transaction** (amount, payee, status, FRAUD/SAFE label, idempotency key), **RiskDecision** (rule, ML and final scores, band, reasons and features as JSON, model version, latency), **Alert**, **LedgerEntry** (every balance change with the balance after it) and **PayeeReport**.

Smaller tables hold rule settings, thresholds, model versions and the audit log. There is a unique index on (user, idempotency key) so a payment cannot be created twice, and indexes on user and payee by time for the history lookups. The Redis keys are listed in Appendix C.

8.6 APIs and Integrations

The API is documented with OpenAPI and can be tried at `/docs`. Table 8.2 lists the main endpoints.

Table 8.2: Main API endpoints

Endpoint	Role	Purpose
`POST /api/auth/register, /login, /logout`	public	Account and session
`GET /api/wallet, /transactions, /safety`	user	Balance, history, warnings and reports
`POST /api/payments/check`	user	Step 1: score the payment, return band, reasons and token
`POST /api/payments/:id/context`	user	Answers to the three questions, re-score
`POST /api/payments/confirm`	user	Step 2: move the money
`POST /api/payments/:id/cancel`	user	Cancel, optionally report
`GET /api/alerts, /alerts/stream; POST /alerts/:id/review`	analyst	Alert queue, live stream (SSE), verdict
`/api/admin/rules, /thresholds, /model, /simulate, /metrics`	admin	Rules, model, simulator, health
`ML: POST /score, /train`	internal	Score 20 features, retrain

Confirm returns 401 if the token is invalid or tampered with, 403 if the payee is a confirmed scam, 409 if the payment was already processed, 422 if the retyped UPI ID is wrong, 425 if the pause is not over yet, and 400 if the balance is too low. In a real bank, the app would call *check* before starting the normal UPI payment, and the bank's own UPI call would take the place of our *confirm*.

8.7 Algorithms and Business Rules

8.7.1 Features

The 20 features are computed by the same logic in TypeScript (for serving) and Python (for training), both reading one shared JSON spec. A test with 300 random inputs checks that both give exactly the same values. The amount z-score compares the payment with the user's last 30 days, $z = (a - \mu) / \max(\sigma, 0.1\mu, 50)$, limited to the range −5 to 20. The floor on σ stops

users with very regular payments from getting huge z-scores. The user's usual payment hour is a circular mean, so 11 pm and 1 am average to midnight and not noon. Velocity counts payments in the last 10 minutes and 1 hour. The mule signal counts how many different people paid the payee in the last 24 hours. The three questionnaire answers are features too, and they stay 0 until answered.

8.7.2 Rules

Table 8.3: Default rules (all editable at runtime)

Rule	Fires when	Points
NEW_PAYEE	User has never paid this UPI ID	15
AMOUNT_SPIKE	z-score ≥ 3, or amount ≥ 2 × 30-day maximum	25
DRAIN_BALANCE	Amount ≥ 80 % of balance	20
VELOCITY_BURST	≥ 3 payments in 10 min or ≥ 6 in 1 h	25
SPLIT_PAYMENTS	≥ 2 similar amounts to the same payee in 1 h	20
ODD_HOUR	Between 11 pm and 6 am, and ≥ 4 h from usual hour	10
MULE_PATTERN	≥ 5 different senders in 24 h and payee ≤ 30 days old	30
REPORTED_PAYEE	At least one report (+5 per extra, max 40)	25+
NEW_DEVICE	Device never used for a payment by this user	15
REPEATED_FAILED_ATTEMPTS	≥ 3 failed logins or re-types in 15 min	20
CONTEXT_PRESSURE	Urged to pay (25), asked for OTP/app (35), unknown recipient (10)	0–70

The rule score is the sum of points (each multiplied by its weight), capped at 100.

8.7.3 Model

We use scikit-learn's HistGradientBoostingClassifier, a gradient boosted tree model [57], with 300 iterations, learning rate 0.08, 31 leaves per tree, L2 regularisation of 1.0 and balanced class weights. Class weighting pushes the probabilities too high, so we calibrate the output with isotonic regression [58, 59] on a separate, later slice of time. The ML score is the calibrated probability × 100. To explain a single decision we use occlusion. We make 20 copies of the payment, each with one feature reset to a typical genuine value, and see how much the fraud probability drops. All 21 rows are predicted in one call, so it is cheap.

8.7.4 Blending and overrides

Final score = 0.4 × rule score + 0.6 × ML score (rule score alone if ML is unavailable)

A final score below 35 is LOW, 35 to 69 is MEDIUM and 70 or above is HIGH. If an analyst has confirmed the payee as a scam, the score is raised to at least 95 and the payment is blocked. If the user says they were asked for an OTP or to install an app, the score is raised to

at least 70. After the questionnaire, the new score is never allowed to be lower than the old one. The reasons are ordered with overrides first, then rules by points, then "matches known scam patterns" if the ML score is 50 or more.

8.7.5 Reliability

The ML call has a 150 ms timeout and a circuit breaker [60]. If it fails 5 times within 30 seconds, we stop calling it for 15 seconds and use rules only. After that, one test request (chosen with a Redis lock) decides whether to start calling it again. The breaker state is in Redis, so all API instances share it. The Idempotency-Key is claimed in Redis with SET NX for 24 hours [61]: a repeat with the same body gets the stored response, a different body gets 422, and a duplicate arriving at the same moment gets 409. Confirm runs in one database transaction with two conditional updates. The first, `status = 'PENDING'`, means only one confirm can win. The second, `balance ≥ amount`, means the balance can never go negative. If anything fails, everything is rolled back.

8.8 Synthetic Data Generation

Since we had no real fraud data, we wrote a generator with a fixed seed (42). It simulates 150 users of the four persona types from Chapter 3 (50 salaried, 40 students, 30 shopkeepers, 30 seniors) over 120 days. The first 30 days are only used to build up history, and the remaining 90 days give about 165,000 payments. There are 1,500 merchants with a Zipf popularity curve, and 20 % of them open during the simulation.

We spent a lot of effort on genuine payments that look suspicious, because without them any detector looks better than it is. These include rent of up to ₹25,000 once a month, two festival periods with bigger and more frequent payments and pop-up stalls paid by many people, first purchases from new shops, shopkeeper supplier batches, students paying late at night, new phones and login typos. About 2.4 % of payments are fraud, spread over four scenarios: impersonation (new payee, amount 3 to 40 times usual, follow-up "deposits"), mule collection (one account collecting a fixed fee from 10 to 28 victims in 6 to 36 hours), velocity drain (4 to 9 similar payments in minutes after a remote-access app is installed) and account takeover (failed logins, a new device, mostly at night, most of the balance). Some scam accounts are old (30 to 40 % are 60 to 900 days old), like bought or rented accounts would be. Victims report the payee 10 to 50 % of the time.

Events are replayed in time order, so each feature only uses what was known before that payment. The data is split by time: the first 60 % for training, the next 15 % for calibration and the last 25 % for testing. A random split would leak future behaviour into training.

CHAPTER 9

Implementation

9.1 What We Built

All the code is in one repository. TypeScript packages are managed as npm workspaces and the ML service is a separate Python package. There are five main parts.

Shared risk core (`packages/shared`). This is a small library with no side effects. It turns inputs into the 20 features, runs the rules, blends the scores, applies the overrides and builds the reason text. Since it is pure logic, it was easy to test thoroughly, and both the API and the browser use exactly the same code. The main decision function is in Appendix D.

ML service (`services/ml`). On first start it generates the data, trains the model and a logistic regression baseline, calibrates, evaluates all three systems on the test period, and saves a versioned model folder before it starts serving. The `/score` endpoint rejects requests that do not have exactly the 20 expected features, and returns the probability, the score, the model version and the top contributing features. Retraining runs in the background. Analyst labels are only ever added to the training data, never to the test data.

API (`apps/api`). The feature builder fetches everything it needs with one PostgreSQL query plus a batch of Redis reads running in parallel. The risk engine calls the shared core and the ML client, saves the decision with its latency and model version, and queues a job. The payments service handles the idempotent check, re-scoring after the questions, cancel and report, and the atomic confirm. Middleware handles sessions, roles and rate limits, and every route validates its input.

Worker. This creates alerts for MEDIUM and HIGH decisions and publishes them for the live analyst view. It also refreshes a user's baseline after each payment, recalculates payee reputation, writes a metrics summary every 30 seconds (which also tells the health page that the worker is alive), syncs the active model every minute and runs retraining when asked. Failed jobs are retried, and if the queue is down, payments still go through.

Web app (`apps/web`). For users, it has the wallet, the Send money flow with the risk meter, reasons, questions and the MEDIUM/HIGH screens, and a My Safety page. Analysts get a live alert queue and a detail view. Admins get the scam simulator, the rules editor with what-if preview, the model evaluation page and the system health page.

9.2 Development Method

We worked in seven short phases, P0 to P6, and each ended with something working, tested and committed. Before writing any code we agreed on a "definition of done", which included the evaluation rule from Chapter 1: the hybrid had to beat rules alone on recall at the same false alarm rate, and if it did not, we would say so. We did the riskiest part (data, model and evaluation) first.

Table 9.1: Development phases

Phase	What was done
P0	Repository setup, Docker Compose with six services, database schema, health endpoints
P1	Data generator, training with calibration, evaluation, scoring service, Node/Python parity tests
P2, P3	Login and roles, feature builder, rules + ML + blend, two-step payments, idempotency, circuit breaker, worker jobs
P4, P5	Wallet, Send money with all three bands, My Safety, analyst console, simulator, rules editor, model and health pages
P6	Worker tests, browser tests, load test, screenshots, documentation

The core build was done in one intensive week starting on 14 October 2026. After that came load testing, fixing the data leaks we found (Section 11.1), demo practice and this report (Figure 9.1).

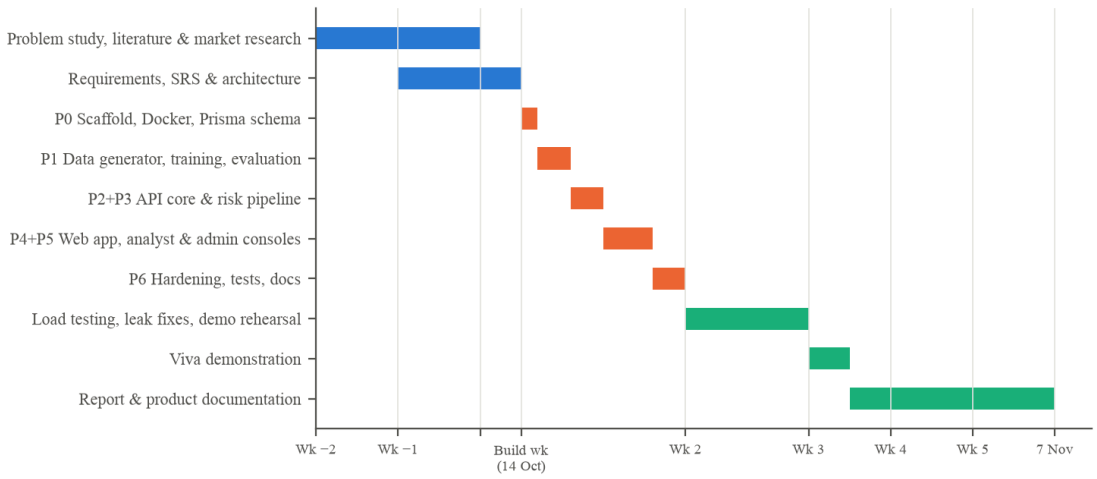


Figure 9.1: Project timeline

9.3 Team Roles

TEAM TO COMPLETE — This split is only a suggestion based on the parts of the system. Replace it with who actually did what.

Table 9.2: Team roles

Member	Main responsibility
Ansh (23BCS016)	Data generator, model training and calibration, evaluation, ML service
Harshit (23BCS054)	API, feature builder, risk engine, payment consistency, worker
Vijender (23BCS132)	Web app and warning screens, analyst and admin consoles, Docker, browser tests
Sourabh (23BEC055)	User research, market and legal study, load testing, test plan, documentation

9.4 Screenshots

These were taken from the running system with the demo data.

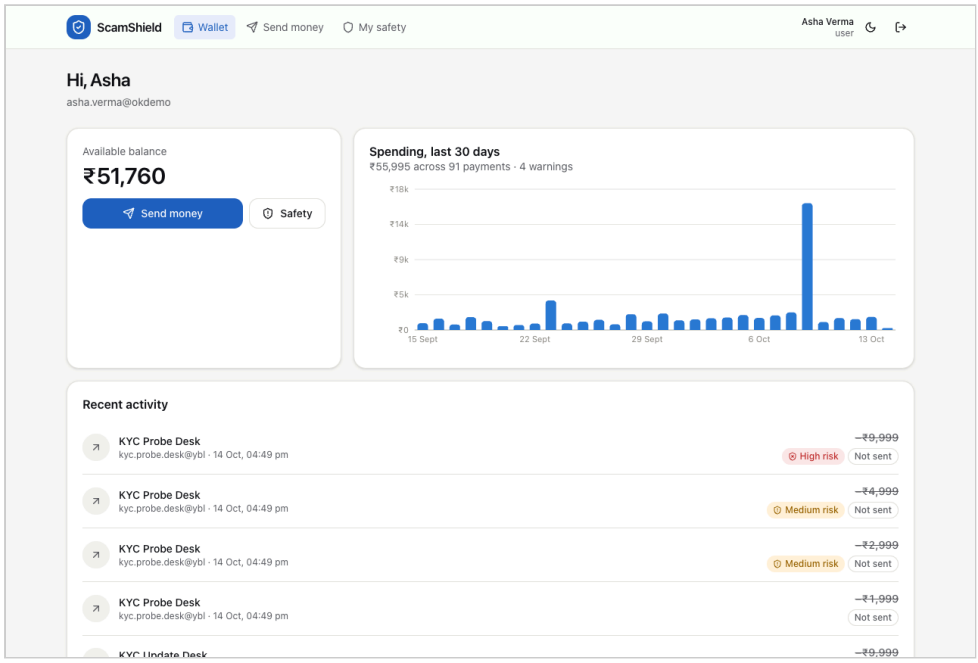


Figure 9.2: Wallet dashboard

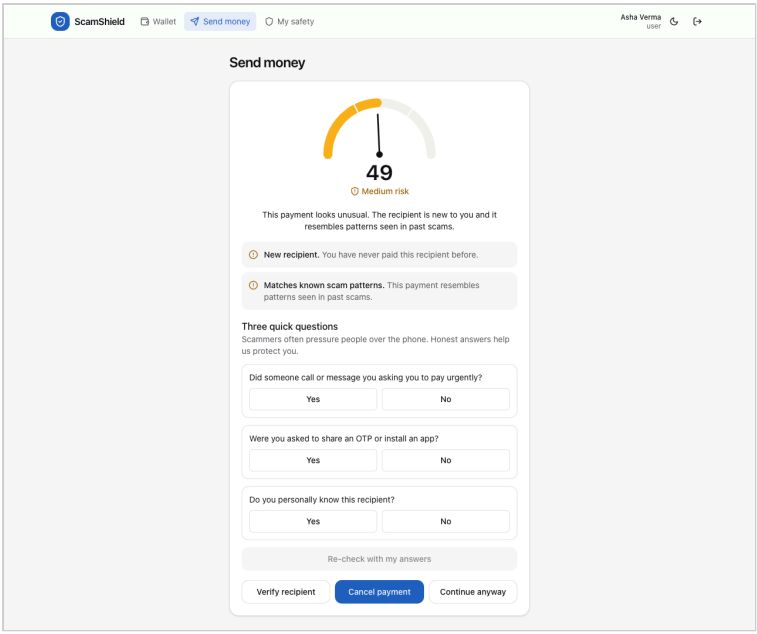


Figure 9.3: A MEDIUM risk payment with reasons and the three questions

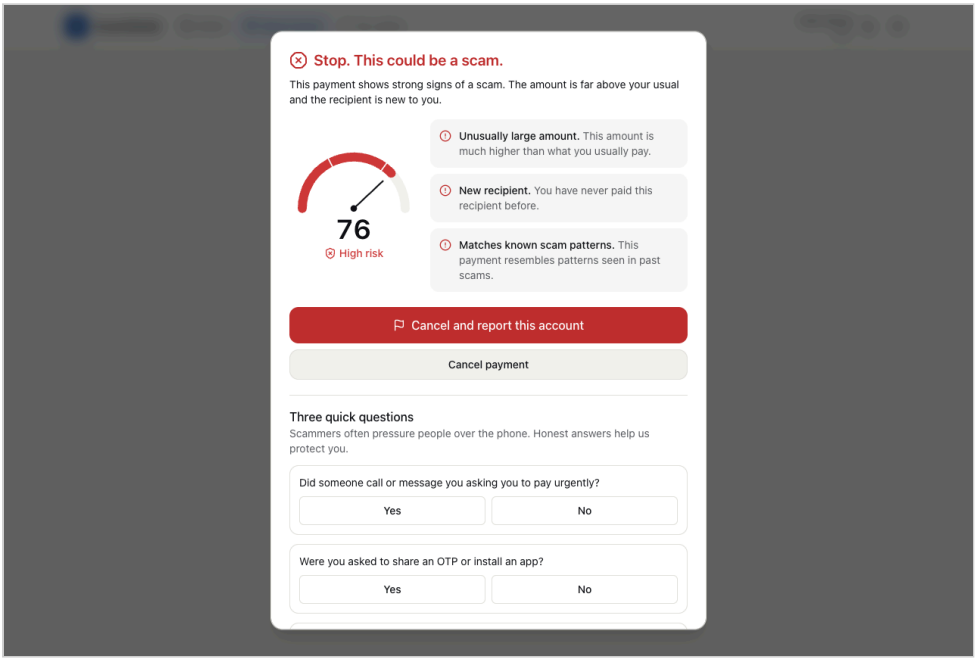


Figure 9.4: A HIGH risk payment, with "Cancel and report this account" as the main button

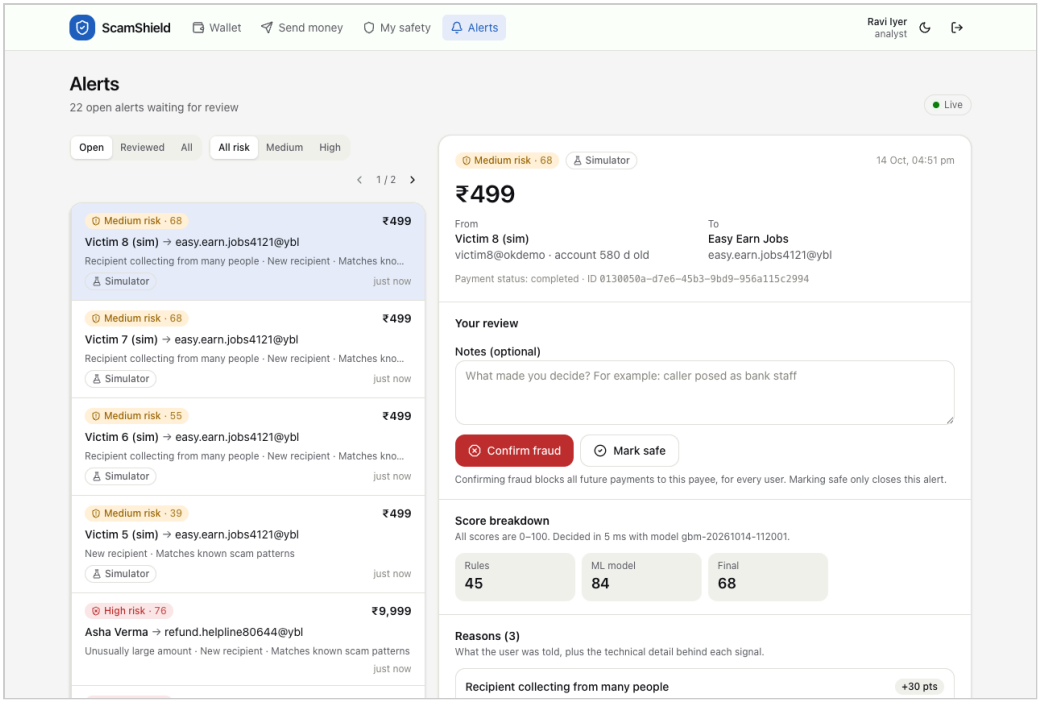


Figure 9.5: Analyst console with the live alert queue and alert detail

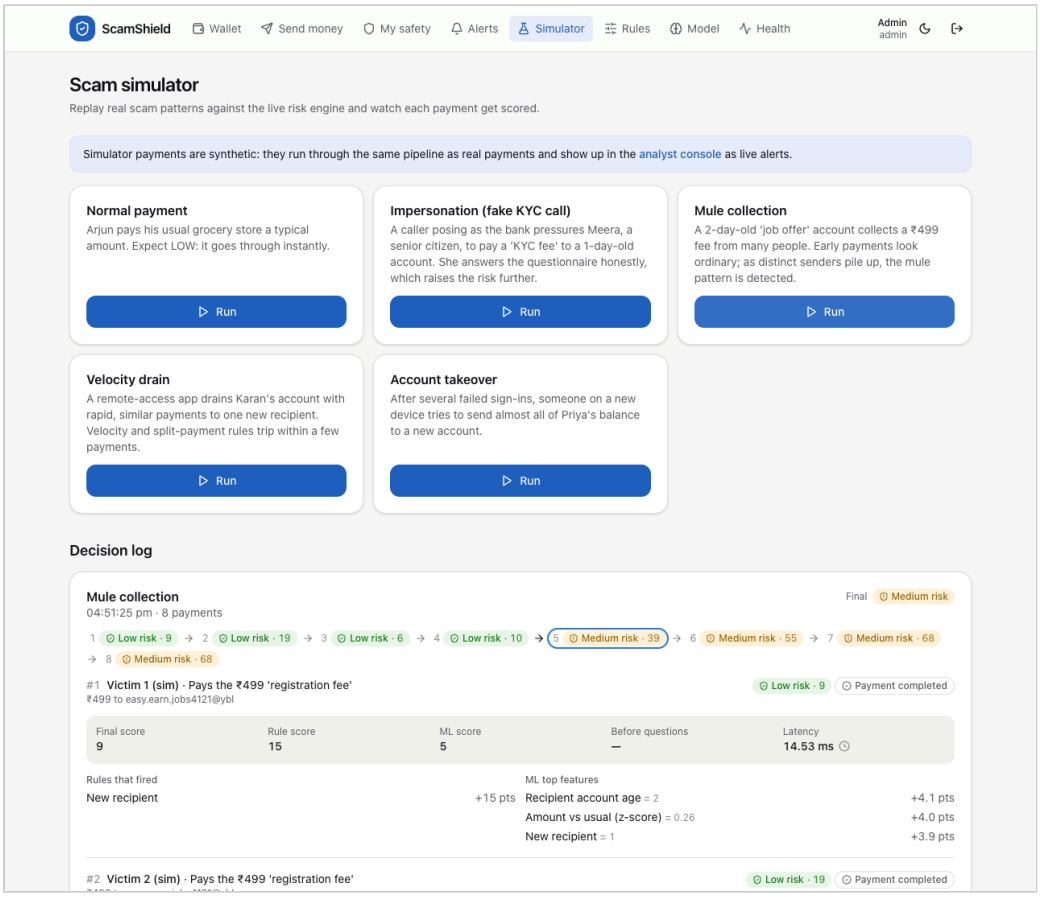


Figure 9.6: Scam simulator: a mule collection run moving from LOW to MEDIUM as more people pay the same account

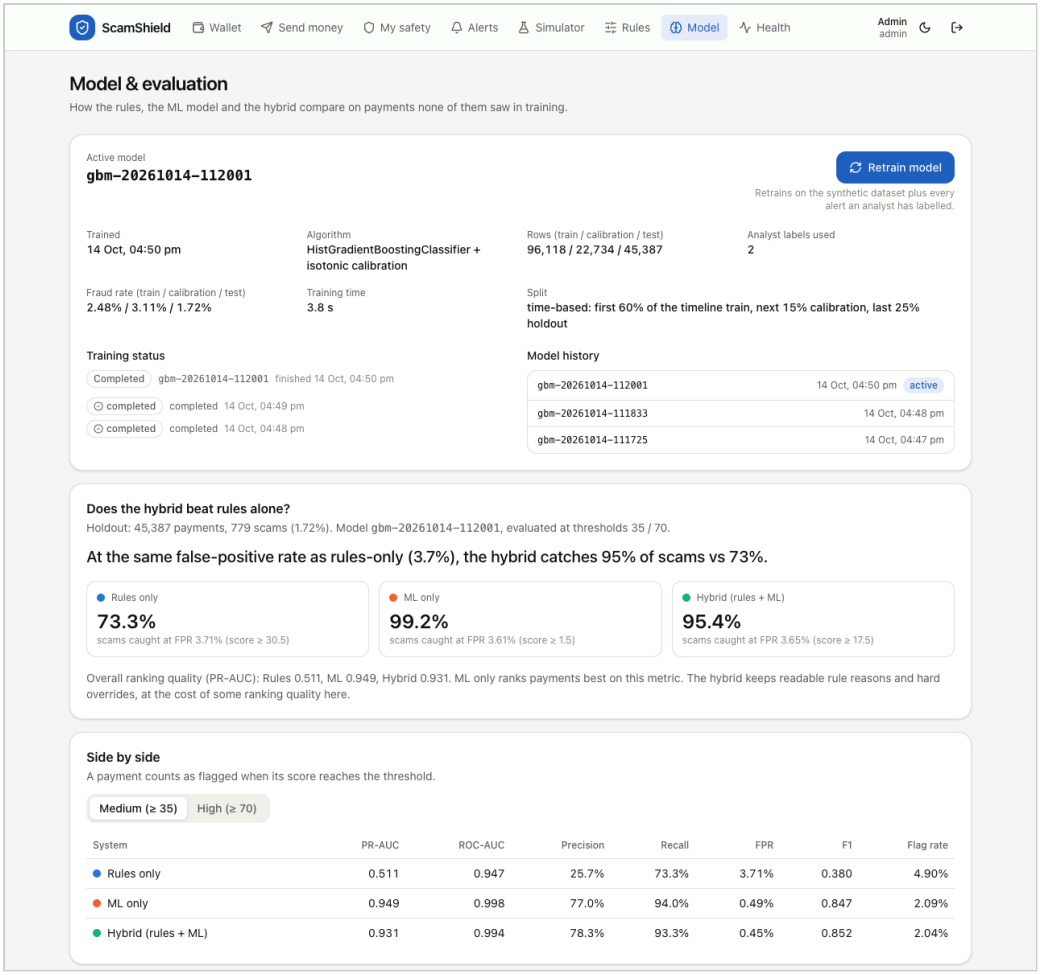


Figure 9.7: Model and evaluation page

The Model page screenshot was taken after a retraining in the app that included two analyst labels, so its numbers are slightly different from the reference model reported in Chapter 10.

CHAPTER 10

Testing and Results

10.1 Testing Strategy

We tested at several levels. Unit tests cover the shared risk core (features, rules, blending, bands, overrides, reason text). A parity test runs 300 random inputs through both the Python and TypeScript feature code and checks they match exactly. Integration tests run the API against real PostgreSQL and Redis with a fake ML service, because things like double spending and the circuit breaker only show up with real databases. These cover login and roles, the feature builder, the two-step payment, idempotency, parallel confirms, tampered tokens, the pause, the questions, blocking, reports, ML failure and worker jobs. The ML service has its own tests for the scoring contract, input checks and speed. Two Playwright browser tests cover a normal payment and a scam payment from start to end, and we load tested the check endpoint with autocannon.

On 7 November 2026 we ran the unit, parity, integration and ML suites again on a fresh copy of the code, and all 341 tests passed (309 in the shared package, 24 API integration tests and 8 ML tests). The two browser tests need the whole Docker stack running, and they last passed during phase P6. The load test was run on 14 October 2026.

10.2 Test Cases

Table 10.1: Main test cases (all passed)

ID	Test	Expected result
TC-01	Normal payment to a known shop	LOW, no prompt, completed with a ledger entry
TC-02	₹9,600 to a 1-day-old new payee, ML score 60	AMOUNT_SPIKE and NEW_PAYEE fire, final 52, MEDIUM, correct summary sentence
TC-03	Answer "no" to every question on a MEDIUM payment	Score does not go down
TC-04	Say yes to "asked to share an OTP"	At least HIGH
TC-05	Confirm HIGH payment immediately, then with wrong UPI ID	425, then 422 and failed attempt counted
TC-06	Pay an analyst-confirmed scam account	BLOCKED, confirm returns 403
TC-07	Repeat check with same key, then with changed body	Same response replayed, then 422; one row only
TC-08	Five confirms of one payment at the same time	Exactly one succeeds, balance correct

TC-09	Tampered or another user's check token	401
TC-10	ML service errors or takes longer than 150 ms	Rules-only decision marked degraded, still within time
TC-11	Five ML failures, then ML recovers	Breaker opens, skips ML, tests once, closes
TC-12	A fraud-labelled payment in history	Left out of the user's baseline
TC-13	User or analyst calls an admin endpoint	403
TC-14	ML /score with a missing or unknown feature	422
TC-15	300 random inputs through Python and TypeScript	Identical features and rule scores
TC-16	Browser: fake KYC payment as Asha	HIGH screen, pause enforced, cancel and report works

10.3 Model Results

The reference model was trained on 96,118 payments and calibrated on 22,734. It was then tested on the last 25 % of the timeline: 45,387 payments, of which 779 (1.72 %) are scams. This includes a festival period it never saw during training. With so few scams, accuracy and ROC-AUC look good for almost any model, so we mainly use precision, recall, false positive rate (FPR) and the area under the precision-recall curve (PR-AUC) [62].

Table 10.2: Results on the test period at the MEDIUM threshold (score ≥ 35)

System	PR-AUC	Precision	Recall	FPR	F1
Rules only	0.511	25.7 %	73.3 %	3.71 %	0.380
ML only	0.946	72.7 %	94.6 %	0.62 %	0.822
Hybrid	0.933	79.1 %	93.2 %	0.43 %	0.856

The main check from our definition of done was recall at the same false alarm rate. Rules alone run at an FPR of 3.71 %. If we set the hybrid's threshold so its FPR is about the same (3.65 %), it catches 745 of the 779 scams (95.6 %), compared with 571 (73.3 %) for rules alone. So the hybrid passed.

It did not win on everything, though. ML alone ranks payments a bit better than the hybrid (PR-AUC 0.946 against 0.933) and has slightly higher recall at 35. Adding the rules costs a little ranking quality. What we get back is readable reasons, the hard overrides, a system that still works when ML is down, and at our chosen thresholds the best precision and the fewest false alarms. At the HIGH threshold of 70 the hybrid's precision is 96.0 % with an FPR of

only 0.056 %, which is why we felt comfortable adding strong friction there. We think this is the right trade for a system that interrupts real people, but it is a trade-off, and the Model page in the app shows it.

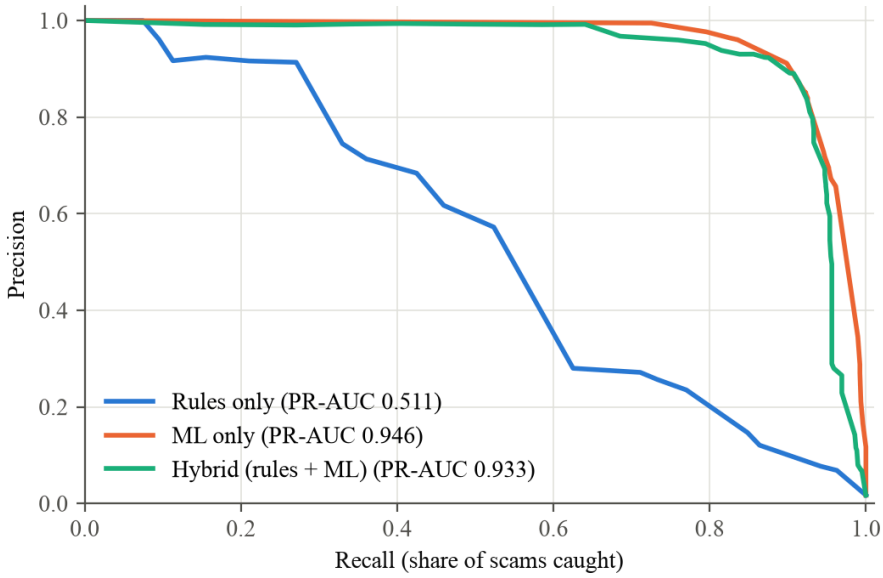


Figure 10.1: Precision-recall curves on the test period



Figure 10.2: Share of scams caught at the MEDIUM threshold, by scenario

By scenario, the hybrid catches 91 % of impersonation, 99 % of mule collection, 89 % of velocity drain and 99 % of account takeover payments (Figure 10.2). Rules alone do worst on account takeover (57 %), where it is the combination of signals that matters. On false alarms, the hybrid flags 0.04 % of everyday payments and 1.0 % of festival payments. It flags none of the shopkeeper supplier batches, while rules alone flag 11.6 % of them. Its weak spot is first payments to new contacts (21.7 %, out of 60) and new merchants (31.3 %, out of 16). These are small groups, but they are the genuine payments that look most like scams.

Calibration improved the Brier score from 0.0046 to 0.0035. The calibration curve shows the model is over-confident between about 30 % and 60 % on the test period, probably because of the festival period, and those ranges have few payments. The most important features (Figure 10.3) are payee age, amount z-score, number of different senders to the payee in 24 hours, and repeated similar amounts. These are reasonable signals, the same kind real fraud teams look at.

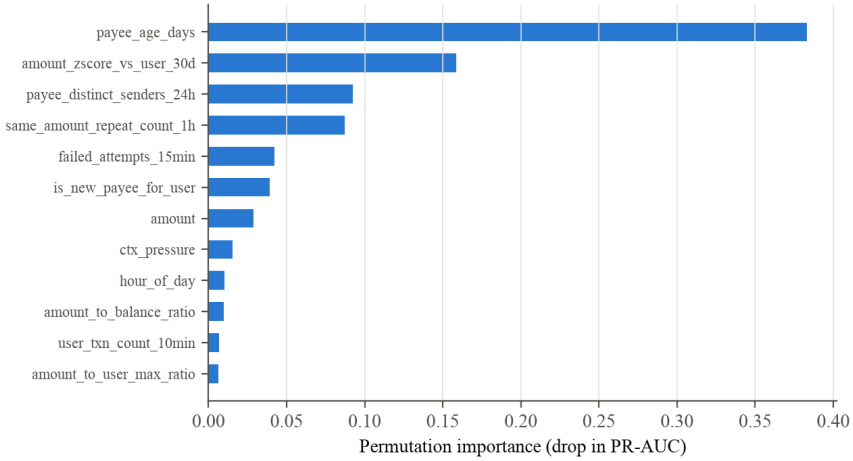


Figure 10.3: Permutation feature importance (top 12 features)

10.4 Performance

We sent 1,200 check requests at 20 per second over 60 seconds. Every one ran the full pipeline with ML, and there were no errors or fallbacks. Measured from the client, the median was 42 ms, the 99th percentile 118 ms and the slowest 147 ms. The API's own metrics gave a p95 of 109 ms, well under the 300 ms target. The first time we ran this test it went badly: the ML service could only do about 47 requests a second, the circuit breaker kept opening and most decisions fell back to rules. The cause was scikit-learn's thread pool, which costs more to start than a single prediction takes. Setting `OMP_NUM_THREADS=1` took the ML service to about 154 requests a second, and the breaker stayed closed. The 20 per second limit came from our per-user rate limit and the 12 test accounts. It is not the most the system can handle.

10.5 User Feedback

We planned user acceptance tests around the four personas. Each participant makes three normal payments, pays a "KYC desk" while one of us acts as a pushy caller, pays a large amount to a known person (rent), and then reviews the alerts as an analyst. A session passes if the normal payments go through with no prompt, the participant can explain in their own words why the payment was flagged, they cancel the coached scam payment, and they find the HIGH screen acceptable.

TEAM TO COMPLETE — Add the results of any user testing sessions and the feedback from the viva demo: how many people, who they were, whether they cancelled the scam payment, and what they said. Only include sessions that actually happened.

CHAPTER 11

Challenges and Limitations

11.1 Problems We Faced

Results that were too good. The first version of our data generator gave a PR-AUC of about 0.99, which was suspicious. Feature importance showed two problems. Scam amounts came from a short fixed list (₹4,999 and similar) and genuine amounts never ended in 99, so the model had simply memorised amounts. Also, every scam account was a few days old while almost every genuine payee was years old, so "young payee" meant scam. We fixed the generator: genuine and scam amounts now use the same price logic, and we added young genuine payees (new shops, friends joining, festival stalls) and old bought scam accounts. PR-AUC came down to about 0.95, which is much more believable.

Polluted baselines. Each simulated user was being hit by many scams, and those scam amounts became part of their "normal" history, so later an ordinary ₹500 payment looked extreme. We now leave fraud-labelled payments out of baselines in both the generator and the API, and give drained balances back after a few days.

Python and TypeScript giving different answers. Features are computed in Python for training and TypeScript for serving. Small differences in rounding or in how IST hours are worked out would quietly feed the model wrong inputs. We moved the definitions into one JSON spec, copied JavaScript's rounding into the Python code, and added the 300-case parity test.

Slow ML under load. This was the thread pool issue described in Section 10.4, fixed with one environment variable. It was good to see that the fallback to rules worked as designed even before the fix.

Double taps and retries. A double tap or a network retry could have charged twice. Idempotency keys, a token tied to amount and payee, and conditional SQL updates in one transaction fixed this, and we tested it with five confirms at the same time.

Repeatable demos. Every demo run changed balances and counters, so the next run behaved differently. We gave the simulator its own set of accounts and reset their state before each run.

11.2 Current Limitations

- Everything is tested on synthetic data. The numbers show that the method works on a realistic simulation, not how it would do on real UPI traffic. The fraud rate in our data (about 2.4 %) is much higher than real life, so real precision would be lower.

- Our simulated scammers do not adapt. Real ones change tactics when they get blocked.
- There is no connection to NPCI, a bank or real money.
- The questions depend on honesty. A coached victim can say "no" to everything. The score will not go down, but the lie is not caught either.
- First payments to new contacts and new merchants are flagged too often (20 to 30 % in small samples).
- The model is over-confident in the 30 to 60 % range during the festival period.
- The confirmed-scam override and analyst labels are not tested offline, since the synthetic data has no analyst labels.
- Everything runs as one instance and was only tested to about 20 requests a second on a laptop.
- Someone could create many accounts to report a payee. Reports never block on their own, but this is not prevented.
- Baselines update in the background, so in a quick burst of payments a few may be scored against a slightly old baseline.
- No penetration test or audit has been done, and the secrets are demo defaults.
- We have not yet run proper user testing (Section 10.5).

CHAPTER 12

Future Scope and Roadmap

12.1 Planned Improvements

The most important next step is testing on real data. We would run ScamShield in shadow mode with a partner, scoring real payments without showing anything to users, then compare the scores with confirmed fraud cases and re-tune the thresholds and calibration. On the detection side we want to use the network of who pays whom to find mule rings, add device fingerprinting, and use outside signals like DoT FRI for the payee's phone number. We also want to watch for drift and get alerts when the model starts to go stale. For safer model updates, a new model should first score live traffic in the background (shadow deployment) and only replace the old one once it is proven.

For the warning screens, we would like to test different wordings with real users. We would also like to add a "trusted contact" confirmation for senior citizens on HIGH payments, vary the warning design so people do not get used to it [20], and add Hindi and Kannada with voice read-out. For analysts, it would help to group cases by payee and export them to the bank's own systems. Federated learning could let banks share fraud patterns without sharing customer data.

12.2 Scaling Plan

The API and ML services keep no state of their own, so we can run more copies behind a load balancer. For a pilot at a few hundred requests a second, three API and three ML copies with managed PostgreSQL (plus a read replica) and Redis in an Indian region should be enough. At a few thousand requests a second we would move to a Redis Cluster with each user's keys kept together, split the transaction tables by month, and autoscale the ML service. At national scale it would make sense to move to Kafka for events, add a proper feature store so that training and serving always match, and run in more than one region.

12.3 Product Roadmap

Table 12.1: Product roadmap

Period	Plan
Short term (0 to 6 months)	Shadow-mode pilot with one partner, proper user research and testing, threshold tuning on real data, Hindi and Kannada warnings, security audit, DPDP readiness
Mid term (6 to 18 months)	Mule network detection, model shadow and canary rollout, Android and iOS SDK, DoT FRI and Suspect Registry signals, second and third customers
Long term (18 to 36 months)	Federated learning across institutions, RBI sandbox participation, deployment across regions at national scale

CHAPTER 13

Product Launch Documentation

13.1 PRD Summary

Table 13.1: Product requirements summary for a pilot release

Item	Details
Product	Pre-payment scam risk engine with warning screens for payers and consoles for analysts and admins
Customers	Cooperative banks, small finance banks and smaller UPI apps
Users	Payers (the four personas), fraud analysts, risk admins
Goals	Fewer successful scam payments, almost no friction on genuine ones, and faster blocking of scam accounts
Not in v1	Blocking calls or SMS, card or net banking fraud, automatic blocking without an analyst (except confirmed scam payees)
Release criteria	In shadow mode on partner data: precision of at least 50 % at HIGH and an FPR of at most 0.5 % at MEDIUM. Also p95 under 300 ms at pilot load, no critical security findings, and a consent flow approved by the partner
Open questions	Who is liable for a missed scam? How do blocked account owners appeal? Which languages come first?

13.2 Lean Canvas

Figure 13.1 puts the business model on one page in the Lean Canvas format [63].

<table>
<tr>
<td rowspan="2">PROBLEM
Victims are talked into paying scammers
Checks verify identity, not intent
Money is rarely recovered
ALTERNATIVES TODAY
Bank FRM tools, 1930 helpline, generic SMS warnings</td>
<td rowspan="2">VALUE PROPOSITION
Stop the scam before the money moves. Tell the payer why, and slow them down only when it matters.
IN SHORT
A smoke alarm for payments.</td>
<td rowspan="2">CUSTOMERS
Cooperative and small finance banks
Smaller UPI apps
EARLY ADOPTERS
Banks with rising scam complaints and no data science team</td>
<td>SOLUTION
Risk check before payment
Warnings matched to risk, with reasons
Analyst feedback and payee blocking</td>
<td>UNFAIR ADVANTAGE
Payer-side warning design and an evaluation anyone can rerun
Cheap enough for small banks</td>
</tr>
<tr>
<td>KEY METRICS
Scams stopped
False alarm rate
Time to block a payee</td>
<td>CHANNELS
Direct to bank risk teams
Core banking and UPI switch vendors
Sandboxes, fintech events</td>
</tr>
<tr>
<td colspan="2">COSTS
Hosting in India, audits, legal
Team, onboarding</td>
<td colspan="3">REVENUE
₹0.01 to 0.02 per check, monthly console subscription, onboarding fee, on-premise licence</td>
</tr>
</table>

Figure 13.1: Lean Canvas for ScamShield

13.3 Cost Estimate

The project itself cost nothing, since we used open-source tools on our own laptops. Table 13.2 estimates the first year of a pilot with one partner. These are rough planning figures based on typical prices, not quotes.

Table 13.2: Estimated first-year pilot budget (₹)

Item	Per year
Two cloud VMs for API, worker and ML (India region)	2,90,000
Managed PostgreSQL with replica and backups	1,80,000
Managed Redis	96,000
Monitoring and log storage (180 days)	84,000
Security audit and penetration test	4,00,000
Legal and compliance review	3,00,000
Team stipends (4 × ₹30,000 per month)	14,40,000
Domain, tools, travel	60,000
Contingency (10 %)	2,85,000
Total	**≈ 31,35,000**

Leaving out the team, the costs are about ₹14 lakh a year. At 2 paise a check, that is covered by about 7 crore checks a year, or roughly 2 lakh a day, which a mid-sized cooperative bank or app could reach. For funding, we could look at the institute incubator, Startup India seed schemes and fintech challenges.

13.4 Risks and Mitigation

Table 13.3: Main risks

Risk	Likelihood / impact	What we would do
Works worse on real data	High / high	Shadow mode first, re-tune on real labels; the rules still give a baseline
Too many false alarms	Medium / high	Nothing shown for LOW, track FPR per user group, preview rule changes, switch off a rule quickly
Scammers coach victims past warnings	High / medium	Answers only raise risk, OTP override, payee signals that do not depend on the victim
Data breach	Low / high	Collect less, encrypt, control access, incident plan, host in India
ML service outage	Medium / medium	Timeout, circuit breaker, rules-only mode (already built)
Big players copy the idea	High / medium	Focus on small institutions and work alongside existing FRM tools
Fake mass reporting	Medium / low	One report per account; only analysts can block

13.5 KPIs After Launch

Table 13.4: KPIs and pilot targets

KPI	Meaning	Target
Scam catch rate	Share of confirmed scams flagged MEDIUM or HIGH	≥ 70 %
False alarm rate	Share of genuine payments flagged at MEDIUM	≤ 0.5 %
HIGH precision	Share of HIGH decisions that are confirmed scams	≥ 50 %
Cancel rate on HIGH	Share of HIGH payments the user cancels	≥ 60 %
Time to block	First victim payment to analyst FRAUD verdict	median ≤ 2 h
Latency	p95 of the check endpoint	< 300 ms
Rules-only share	Decisions made without ML	≤ 0.1 %
User feedback	"The warning was helpful" in a short survey	≥ 70 % agree

CHAPTER 14

Conclusion

We started with a simple observation. Most UPI fraud happens because people are convinced to send money themselves, and PINs and OTPs cannot stop that. Our answer was to check each payment in the short gap after the user presses Pay and before the money moves, and to tell the user what we see.

ScamShield does this with 20 behavioural features, 11 rules and a calibrated gradient boosting model. On a synthetic test period of 45,387 payments that the model never saw, the hybrid caught 95.6 % of scams at the false alarm rate where rules alone caught 73.3 %. At our chosen thresholds it also had the best precision of the three systems we compared. ML alone ranked payments slightly better, and we found and fixed two leaks in our own data that had made the first results look almost perfect. We have reported both of these, because we think they matter as much as the headline number.

The rest of the system is built around how people actually behave. Most payments see no warning at all. Risky ones get a short, specific explanation and three questions, and the riskiest get a pause and a re-typed UPI ID, with "Cancel and report" as the easiest choice. When analysts confirm a scam account, it is blocked for everyone. The system met its performance target (p95 around 110 ms against 300 ms), keeps working when the ML service fails, and passes 341 automated tests.

The biggest open question is how well it works on real data. Our numbers come from a simulation, and the next step is a shadow-mode pilot with a real partner. Still, we think the project shows that a payment app can do more than ask for a PIN. It can notice when something is off, explain it in plain words, and give the user a moment to think before the money is gone.

References

References are listed in IEEE style in order of first citation. Government statistics from parliamentary replies are cited through the published reports that carried them; readers requiring the primary record should consult the corresponding Lok Sabha / Rajya Sabha question on the Digital Library of Parliament (eparlib.sansad.in).

1. Free Press Journal, "UPI transaction volume rises nearly 13,000-fold in a decade to over 24,162 crore: Govt" (reporting a Lok Sabha written reply citing NPCI data). [Online]. Available: https://www.freepressjournal.in/business/upi-transaction-volume-rises-nearly-13000-fold-in-a-decade-to-over-24162-crore-govt. Accessed: Nov. 7, 2026.
2. Outlook Money, "UPI transaction value reaches Rs 314 lakh crore in FY26, Govt says in Lok Sabha." [Online]. Available: https://www.outlookmoney.com/banking/upi-transaction-value-reaches-rs-314-lakh-crore-in-fy26-govt-says-in-lok-sabha. Accessed: Nov. 7, 2026.
3. EaseMyPrep News, "Nearly 55.49 crore users onboarded on UPI as in June 2026" (reporting a Lok Sabha written reply, Ministry of Finance), Jul. 2026. [Online]. Available: https://easemyprep.in/news/3248-nearly-5549-crore-users-onboarded-on-upi-as-in-june-2026. Accessed: Nov. 7, 2026.
4. National Payments Corporation of India, "UPI product statistics." [Online]. Available: https://www.npci.org.in/product/upi/product-statistics. Accessed: Nov. 7, 2026.
5. YourStory, "UPI frauds peak in FY24, show signs of decline: Parliament data," Dec. 2025 (reporting Lok Sabha Unstarred Question 2481, Ministry of Finance). [Online]. Available: https://yourstory.com/2025/12/upi-frauds-peak-in-fy24-show-signs-of-decline-parliament-data. Accessed: Nov. 7, 2026.
6. YourStory, "Citizens lost over Rs 22,845 crore to cyber criminals in 2024," Jul. 2025 (reporting a Lok Sabha written reply, Ministry of Home Affairs). [Online]. Available: https://yourstory.com/2025/07/citizens-lost-over-rs-22845-cr-to-cyber-criminals-in-2024. Accessed: Nov. 7, 2026.
7. Business Standard, "One in five UPI users' families hit by transaction fraud since 2022: LocalCircles survey," Jun. 26, 2025. [Online]. Available: https://www.business-standard.com/finance/news/upi-transaction-fraud-india-survey-one-in-five-users-hit-localcircles-125062601141_1.html. Accessed: Nov. 7, 2026.
8. Free Press Journal, "Centre blocks 3,718 fraudulent apps, saves Rs 11,158 crore from cyber fraud," Aug. 12, 2026 (reporting a Lok Sabha written reply, Ministry of Home Affairs). [Online]. Available: https://www.freepressjournal.in/india/centre-blocks-3718-fraudulent-apps-saves-11158-crore-from-cyber-fraud. Accessed: Nov. 7, 2026.
9. R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Statistical Science*, vol. 17, no. 3, pp. 235–255, 2002, doi: 10.1214/ss/1042727940.
10. A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," *Expert Systems with Applications*, vol. 41, no. 10, pp. 4915–4928, 2014, doi: 10.1016/j.eswa.2014.02.026.
11. IANS, "UPI frauds worth Rs 805 crore witnessed this fiscal so far: Minister," Dec. 15, 2025. [Online]. Available: https://ianslive.in/upi-frauds-worth-rs-805-crore-witnessed-this-fiscal-so-far-minister--20251215175703. Accessed: Nov. 7, 2026.

12. Inc42, "Indians lost INR 1,935 Cr to digital arrest scams in 2024: Govt," Mar. 2025 (reporting a Rajya Sabha reply). [Online]. Available: https://inc42.com/buzz/indians-lost-inr-1935-cr-to-digital-arrest-scams-in-2024-govt. Accessed: Nov. 7, 2026.
13. NewsMeter, "51 percent of UPI fraud victims never filed a complaint: LocalCircles survey." [Online]. Available: https://newsmeter.in/data-stories/51-percent-of-upi-fraud-victims-never-filed-a-complaint-localcircles-survey-750856. Accessed: Nov. 7, 2026.
14. MediaNama, "Cards and internet frauds made up 66% of bank fraud cases in FY25: RBI," Dec. 30, 2025 (reporting RBI, *Report on Trend and Progress of Banking in India 2024-25*). [Online]. Available: https://www.medianama.com/2025/12/223-cards-internet-fraud-66-in-fy25-decline-banking-rbi/. Accessed: Nov. 7, 2026.
15. F. Stajano and P. Wilson, "Understanding scam victims: Seven principles for systems security," *Communications of the ACM*, vol. 54, no. 3, pp. 70–75, 2011, doi: 10.1145/1897852.1897872.
16. D. Akhawe and A. P. Felt, "Alice in Warningland: A large-scale field study of browser security warning effectiveness," in *Proc. 22nd USENIX Security Symposium*, 2013, pp. 257–272.
17. A. Vance, B. B. Anderson, C. B. Kirwan, and D. Eargle, "Tuning out security warnings: A longitudinal examination of habituation through fMRI, eye tracking, and field experiments," *MIS Quarterly*, vol. 42, no. 2, pp. 355–380, 2018, doi: 10.25300/MISQ/2018/14124.
18. A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: A realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, 2018, doi: 10.1109/TNNLS.2017.2736643.
19. The Paypers, "Lenders reimburse GBP 173 million in APP fraud claims under new regime." [Online]. Available: https://thepaypers.com/fraud-and-fincrime/news/lenders-reimburse-gbp-173-million-in-app-fraud-claims-under-new-regime. Accessed: Nov. 7, 2026.
20. B. B. Anderson, A. Vance, C. B. Kirwan, D. Eargle, and J. L. Jenkins, "How polymorphic warnings reduce habituation in the brain: Insights from an fMRI study," in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, 2015, pp. 2883–2892, doi: 10.1145/2702123.2702322.
21. A. Cooper, *The Inmates Are Running the Asylum*. Indianapolis, IN, USA: Sams Publishing, 2004.
22. MarketsandMarkets, "India fraud detection and prevention market." [Online]. Available: https://www.marketsandmarkets.com/Market-Reports/geography/fraud-detection-prevention-market/India. Accessed: Nov. 7, 2026.
23. IMARC Group, "India fraud detection and prevention market size, share, trends and forecast 2025-2033." [Online]. Available: https://www.imarcgroup.com/india-fraud-detection-prevention-market. Accessed: Nov. 7, 2026.
24. KPMG India, "Reserve Bank of India (Authentication Mechanisms for Digital Payment Transactions) Directions, 2025," Dec. 2025. [Online]. Available: https://kpmg.com/in/en/insights/2025/12/reserve-bank-of-india-rbi-authentication-mechanisms-for-digital-payment-transactions-directions-2025.html. Accessed: Nov. 7, 2026.
25. MediaNama, "23 banks using RBI's MuleHunter.AI to detect mule accounts: RTI," Dec. 2025. [Online]. Available: https://www.medianama.com/2025/12/223-rti-23-banks-mulehunter-mule-accounts/. Accessed: Nov. 7, 2026.
26. Business Today, "Indian govt, RBI Innovation Hub sign AI pact to tackle financial frauds," May 12, 2026. [Online]. Available: https://www.businesstoday.in/technology/story/indian-govt-rbi-innovation-signs-ai-pact-to-tackle-financial-frauds-531090-2026-05-12. Accessed: Nov. 7, 2026.

27. Business Standard, "DoT launches Financial Fraud Risk Indicator to aid cybercrime detection," May 21, 2025. [Online]. Available: https://www.business-standard.com/industry/news/dot-launches-financial-fraud-risk-indicator-to-aid-cybercrime-detection-125052101912_1.html. Accessed: Nov. 7, 2026.
28. Free Press Journal, "Government's anti-cyber fraud drive disconnects 39.4 lakh mobile connections, blacklists 2.27 lakh handsets," Feb. 2026. [Online]. Available: https://www.freepressjournal.in/tech/governments-anti-cyber-fraud-drive-disconnects-394-lakh-mobile-connections-blacklists-227-lakh-handsets. Accessed: Nov. 7, 2026.
29. *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998, 1998, doi: 10.1109/IEEESTD.1998.88286.
30. Government of India, *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023), Gazette of India, Aug. 11, 2023.
31. Storyboard18, "Govt notifies final DPDP Rules, sets staggered rollout with key obligations for data fiduciaries," Nov. 2025. [Online]. Available: https://www.storyboard18.com/digital/breaking-govt-notifies-final-dpdp-rules-sets-staggered-rollout-with-key-obligations-for-data-fiduciaries-84201.htm. Accessed: Nov. 7, 2026.
32. Government of India, *The Information Technology Act, 2000* (Act No. 21 of 2000), as amended by the Information Technology (Amendment) Act, 2008.
33. Ministry of Communications and Information Technology, *Information Technology (Reasonable Security Practices and Procedures and Sensitive Personal Data or Information) Rules, 2011*, G.S.R. 313(E), Apr. 11, 2011.
34. Indian Computer Emergency Response Team (CERT-In), "Directions under sub-section (6) of section 70B of the Information Technology Act, 2000 relating to information security practices, procedure, prevention, response and reporting of cyber incidents for safe and trusted Internet," No. 20(3)/2022-CERT-In, Apr. 28, 2022.
35. Reserve Bank of India, "Storage of Payment System Data," RBI/2017-18/153, DPSS.CO.OD No. 2785/06.08.005/2017-2018, Apr. 6, 2018.
36. European Parliament and Council, "Regulation (EU) 2016/679 (General Data Protection Regulation)," *Official Journal of the European Union*, L 119, pp. 1–88, May 4, 2016.
37. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015, doi: 10.17487/RFC7519.
38. OWASP Foundation, "Application Security Verification Standard 4.0.3," Oct. 2021. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/
39. W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 5, 2023. [Online]. Available: https://www.w3.org/TR/WCAG22/
40. Government of India, *The Rights of Persons with Disabilities Act, 2016* (Act No. 49 of 2016).
41. Reserve Bank of India, "Master Direction on Digital Payment Security Controls," RBI/2020-21/74, Feb. 18, 2021. [Online]. Available: https://www.rbi.org.in/Scripts/BS_ViewMasDirections.aspx?id=12032. Accessed: Nov. 7, 2026.
42. Reserve Bank of India, "Master Directions on Fraud Risk Management in Commercial Banks (including Regional Rural Banks) and All India Financial Institutions," Jul. 15, 2024. Summary: Grant Thornton Bharat. [Online]. Available: https://www.grantthornton.in/globalassets/1.-member-firms/india/assets/pdfs/services/rbi-master-directions-on-fraud-risk-management.pdf. Accessed: Nov. 7, 2026.

43. The Week, "Major UPI changes from August 1: balance enquiry limits, pending payment status and more from NPCI," Aug. 1, 2025. [Online]. Available: https://www.theweek.in/news/biz-tech/2025/08/01/major-upi-changes-from-august-1-here-is-the-full-list-of-balance-enquiry-limits-pending-payment-status-upi-security-and-more-from-npci.html. Accessed: Nov. 7, 2026.
44. Docker, Inc., "Docker Compose documentation." [Online]. Available: https://docs.docker.com/compose/. Accessed: Nov. 7, 2026.
45. Vercel, "Next.js documentation." [Online]. Available: https://nextjs.org/docs. Accessed: Nov. 7, 2026.
46. OpenJS Foundation, "Express — Node.js web application framework." [Online]. Available: https://expressjs.com/. Accessed: Nov. 7, 2026.
47. The PostgreSQL Global Development Group, "PostgreSQL 16 documentation." [Online]. Available: https://www.postgresql.org/docs/16/. Accessed: Nov. 7, 2026.
48. Prisma Data, Inc., "Prisma ORM documentation." [Online]. Available: https://www.prisma.io/docs. Accessed: Nov. 7, 2026.
49. Redis Ltd., "Redis documentation." [Online]. Available: https://redis.io/docs/. Accessed: Nov. 7, 2026.
50. Taskforce.sh, "BullMQ documentation." [Online]. Available: https://docs.bullmq.io/. Accessed: Nov. 7, 2026.
51. S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/. Accessed: Nov. 7, 2026.
52. F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
53. T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
54. G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 3146–3154.
55. S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.
56. Microsoft, "Playwright documentation." [Online]. Available: https://playwright.dev/. Accessed: Nov. 7, 2026.
57. J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001, doi: 10.1214/aos/1013203451.
58. B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2002, pp. 694–699, doi: 10.1145/775047.775151.
59. A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proc. 22nd Int. Conf. Machine Learning (ICML)*, 2005, pp. 625–632, doi: 10.1145/1102351.1102430.
60. M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*, 2nd ed. Raleigh, NC, USA: Pragmatic Bookshelf, 2018.
61. M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017.
62. T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015,

doi: 10.1371/journal.pone.0118432.

63. A. Maurya, *Running Lean: Iterate from Plan A to a Plan That Works*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2012.

APPENDIX A

Questionnaire

Shown first: "This survey is anonymous and voluntary. We do not ask for your name, phone number or account details. Answers are only used for our college project." Respondents under 18 were asked not to continue.

1. Age group: 18–24 / 25–40 / 41–59 / 60+
2. Occupation: Student / Salaried / Self-employed or shop owner / Homemaker / Retired / Other
3. How often do you pay with UPI? Several times a day / Daily / Few times a week / Rarely / Never
4. Which UPI app do you use most?
5. Who set up your UPI app? Myself / Family member / Bank staff / Other
6. In the last 12 months, did you get a call or message you think was a scam? Yes / No / Not sure
7. If yes, what kind? Fake KYC or bank / Fake police or "digital arrest" / Job or task / Investment / Fake buyer or refund / Prize / OTP request / Asked to install an app / Other
8. Have you or a family member lost money in a UPI scam? Me / Family member / No / Prefer not to say
9. If yes, how? Paid myself after being convinced / Shared OTP or PIN / An app was installed / Don't know
10. Was the scammer in touch while paying? Phone or video call / Chat or SMS / Link or QR code / No / Don't know
11. Amount lost: under ₹1,000 / ₹1,000–10,000 / ₹10,000–1 lakh / over ₹1 lakh / Prefer not to say
12. Was money recovered? All / Some / None / Still pending
13. Did you report it? Bank / 1930 or cybercrime.gov.in / Police / Didn't report (why?)
14. Have you ever seen a warning in your payment app before paying? Yes / No / Don't remember
15. If the app thought a payment was very likely a scam, would you accept a 10-second pause? Yes / Only for big amounts / No
16. Would you accept typing the receiver's UPI ID again for such a payment? Yes / Maybe / No
17. Would you answer three yes/no questions before a risky payment? Yes / Maybe / No
18. Which message would make you stop most? (a) "Be careful while paying." (b) "You have never paid this person. Their account is 2 days old and 3 people have reported it." (c) "This payment is risky."
19. Are you OK with your app checking your payment patterns only to catch scams? Yes / Yes, if I can see what is used / No
20. Anything else you want to share?

APPENDIX B

Survey Data

TEAM TO COMPLETE — Fill this sheet from your collected responses. Mention n, dates and how the form was shared, and how many incomplete responses were removed.

Table B.1: Coding sheet for survey responses

Q	Variable	Codes	n	%
Q1	age_group	1 = 18–24, 2 = 25–40, 3 = 41–59, 4 = 60+		
Q6	scam_contact	1 yes, 0 no, 9 not sure		
Q8	lost_money	1 self, 2 family, 0 no, 9 prefer not		
Q12	recovered	1 all, 2 some, 3 none, 4 pending		
Q15	accept_pause	1 yes, 2 big amounts only, 0 no		
Q17	accept_questions	1 yes, 2 maybe, 0 no		
Q18	message_choice	a / b / c		
Q19	privacy_ok	1 yes, 2 yes if transparent, 0 no		

APPENDIX C

Redis Keys

Table C.1: Redis keys used by the API and worker

Key	Expires	Used for
`vel:{user}, vel:{user}:{upi}`	2 h	Payments in the last 10 min / 1 h; similar amounts to one payee
`payee:{upi}:senders`	25 h	Different senders in 24 h (mule signal)
`payee:{upi}:info`	10 min	Cached payee age, reports, confirmed fraud
`user:{id}:payees, :devices`	7 d	Known payees and devices
`user:{id}:profile`	24 h	30-day baseline
`fail:{id}`	20 min	Failed logins and re-types
`idem:{user}:{key}`	24 h	Stored responses for idempotency
`rl:*, cb:ml:*`	seconds	Rate limits and circuit breaker
`scamshield:events`	pub/sub	Live alerts to analyst browsers

APPENDIX D

Code Excerpt

The decision function from `packages/shared/src/blend.ts`, which blends the scores, applies the overrides and orders the reasons:

```
export function decide(input: DecisionInput): DecisionOutcome {
  const { thresholds: t } = input;
  let finalScore = blendScores(input.ruleScore, input.mlScore, t);
  const overrides: OverrideCode[] = [];
  if (input.payeeConfirmedFraud) {
    overrides.push("CONFIRMED_FRAUD_PAYEE");
    finalScore = Math.max(finalScore, CONFIRMED_FRAUD_FLOOR);      // 95
  }
  if (input.features.ctx_otp_or_app === 1) {
    overrides.push("OTP_REQUESTED");
    finalScore = Math.max(finalScore, t.highThreshold);             // at least HIGH
  }
  const reasons: Reason[] = [
    ...overrides.map((code) => ({ code, points: 100, vars: {} })),
    ...input.hits.map((h) => ({ code: h.code, points: h.points, vars: h.vars })),
  ];
  if (input.mlScore !== null && input.mlScore >= ML_REASON_MIN_SCORE) {
    reasons.push({ code: "ML_PATTERN", points: Math.round(input.mlScore * t.mlWeight),
                   vars: { mlScore: Math.round(input.mlScore) } });
  }
  return { finalScore, band: bandFor(finalScore, t), blocked:
input.payeeConfirmedFraud,
           degraded: input.mlScore === null, overrides, reasons };
}
```

APPENDIX E

User Manual

Starting it. Install Docker Desktop and `make`, then run `make demo` in the project folder. The first start takes 3 to 6 minutes because the ML container generates the data and trains the model. Then open `http://localhost:3000`. The API docs are at `http://localhost:4000/docs`. Other useful commands are `make test`, `make e2e`, `make loadtest` and `make reset` (which deletes all data).

Demo accounts (password `Demo@1234`, or use the buttons on the landing page): `asha@demo.scamshield.in` (user), `ravi@demo.scamshield.in` (analyst) and `admin@demo.scamshield.in` (admin). For a quick demo, pay `kirana.sharma@okaxis` (normal), `kyc.update.desk@ybl` (2-day-old "KYC desk"), `olx.seller.deals@ybl` (reported 3 times) or `prize.claim.center@paytm` (confirmed scam, blocked).

Paying. Go to Send money, enter the UPI ID and look at the recipient card, enter the amount and press *Check & pay*. A LOW payment goes straight through. For MEDIUM, read the reasons, answer the three questions and press *Re-check with my answers*, then verify, cancel or continue. For HIGH, the safest option is *Cancel and report this account*. If you really know the person, wait for the 10-second pause and type their UPI ID again. "Payment stopped" means the account is a confirmed scam. My Safety lists your warnings and reports.

Analysts. Open Alerts. New alerts appear live. Click one to see the scores, reasons, model features and payee history, add a note, and choose *Confirm fraud* (this blocks the payee for everyone) or *Mark safe*.

Admins. Use the Simulator to run a scam scenario step by step. Use Rules to change rules and thresholds; the what-if panel shows the effect before you save. Model shows the evaluation and lets you retrain, and Health shows services, latency, ML calls and queues. If a decision says "Checked with safety rules only", the ML service was down; check `make ps`.